SchoolEye ERP — DPDP Act Complete Implementation Guide

For: SchoolEye (V3M School ERP) · **Prepared for:** implementation with Claude Code · **Version:** 2.0 (22 Oct 2026)

Complete, step-by-step plan to make SchoolEye compliant with the **Digital Personal Data Protection Act, 2023** and the **DPDP Rules, 2025**, with:

- **Masking everywhere by default.** Every screen, grid, Excel export and API response shows masked personal data (`98******21`, `XXXX XXXX 1234`).
- **Sensitive data locked in the database.** Aadhaar, health and bank details are encrypted; the key never sits in the database.
- **One controlled place to see everything.** A new **Student Full Details** page lets the **School Admin** pick Session → Class → Section → Student No and see the complete, unmasked profile of the student and parents (photo, mobile, email, address, Aadhaar, login). Every reveal needs a reason, is time-limited and is logged.

> **Stack:** ASP.NET Core Web API · Dapper · SQL Server stored procedures · Blazor Server (InteractiveServer) · `ApiResponse<T>`
> **Disclaimer:** Technical implementation plan, not legal advice. The school's counsel must confirm notice text, retention periods and use of exemptions.

Contents

- Part A — How to use this document with Claude Code
- Part B — DPDP in short
- Part C — Data protection architecture (masking, encryption, reveal)
- Part D — Student Full Details page (School Admin)
- Part E — Full DPDP module, step by step
- Part F — Testing and go-live
- Appendix A — Complete DPDP schema script
- Appendix B — Ready-to-paste Claude Code prompts
- Sources

Part A — How to use this document with Claude Code

Put this file in the repository root as `docs/DPDP_IMPLEMENTATION.md` and tell Claude Code to read it before every phase. The prompts in Appendix B do this.

A.1 Rules Claude Code must follow

1. **The existing project is the source of truth.** Analyse the solution and `script.sql` before writing code. Never treat SchoolEye as greenfield.
2. **Keep the stack.** ASP.NET Core Web API + Dapper + SQL Server stored procedures + Blazor. No Entity Framework.
3. **Layered pattern for every API:** Model → Interface → Service (Dapper) → Controller → Stored Procedure.
4. **One stored procedure per standalone module.** Do not merge unrelated modules into one SP.
5. **Dapper parameters:** plain `param.Add("@Name", value)`. Give `dbType`/`size` only for OUTPUT parameters.
6. **Literal mapping:** model properties map field-for-field to SP parameters and result columns.
7. **Multi-row saves use a TVP** (User-Defined Table Type), never a loop of SQL calls.
8. **Naming:** tables `V3M_DPDP_*`, action SPs `USP_DPDP_*`, list/get SPs `V3M_DPDP_*`, table types `TT_DPDP_*`. Keep existing naming everywhere else.
9. **Keep the existing `ApiResponse<T>` shape**, folder structure and DI registration style.
10. **Check before regenerate.** If an SP, query or model already exists, read it and modify it; do not rebuild from scratch.
11. **Blazor conventions:**
    - No `<label>` in `PageHeading` form-box fields.
    - Only Session / admission-type filters + sliders-icon button + Search in `PageHeading`; all other filters go in the `col-xl-9` search popup.
    - Excel export through `AdmitChildButton` → `Export/Generate*ExcelData` → `downloadExcel` JS.
    - File uploads use `MultipartFormDataContent` through the injected `HttpClient`, not `httpService.Post<T>`.
12. **Deliver each API as its own set of files** (Model, Interface, Service, Controller, SQL) so each can be reviewed separately.
13. **Never print real personal data** in logs, console output, test fixtures, commit messages or comments.
14. **Stop and ask** if a table or column in this document does not exist in `script.sql`. Do not invent schema.

A.2 Placeholders to resolve in Step E1

This document uses the names below. Claude Code must map each one to the real SchoolEye object in Step E1 and replace it everywhere.

Placeholder in this doc	Meaning	Real name (fill in)
`dbo.StudentMaster`	Student master record	
`dbo.StudentSessionMap`	Student ↔ session/class/section/roll	
`dbo.ParentMaster`	Father / mother / guardian	
`dbo.ClassMaster`, `dbo.SectionMaster`	Class and section masters	
`dbo.SessionMaster`	Academic session	
`dbo.MstUsers`	Login table (exists)	`MstUsers`
`StudentNo`	Admission / scholar number	
Claim `RoleId`, `LoginId`, `GroupCode`, `BranchCode`	JWT claim names	
`GroupCode`, `BranchCode`, `StudentId`, `ParentId`, `LoginId`, `SessionId` types	Real SQL types	

Part B — DPDP in short

B.1 Key dates

Date	What happens
13 Nov 2025	DPDP Rules notified (Gazette G.S.R. 846(E)); Data Protection Board provisions in force
13 Nov 2026	Consent Manager registration provisions start
13 May 2027	Notice, consent, children's data, security, breach reporting, rights — **all enforceable**

> The Gazette is dated 13 Nov 2025; PIB's explainer says 14 Nov, and some firms give the final date as 12 May 2027. Plan to go live by **March 2027**.

B.2 Roles

Role	Who
Data Fiduciary	The school — decides why student, parent and staff data is collected
Data Processor	V3M (SchoolEye) — processes on the school's instructions
Data Principals	Parents, students, staff. Every student under 18 is a **child**; the parent acts for them

B.3 What the law requires and where SchoolEye covers it

Requirement	Law	SchoolEye feature
Notice before collecting data	s.5, Rule 3	Versioned privacy notice per branch and language
Consent per purpose, withdrawable	s.6	Consent register, one row per purpose
Legitimate uses (no consent)	s.7	Legal-basis flag on each purpose
Verifiable parental consent	s.9(1), Rule 10	Parent verification before consent
No tracking / targeted ads for children	s.9(3)	No ad / analytics trackers on student pages
School exemption	Fourth Schedule	Only for educational activities and child safety
Guardian for persons with disabilities	Rule 11	Lawful guardian record
Reasonable security safeguards	s.8(5), Rule 6	**Encryption, masking, controlled reveal, access log** (Part C)
Breach intimation	s.8(6), Rule 7	Breach register; detailed Board report within 72 h
Erase when purpose ends	s.8(7), Rule 8	Retention policies, 48-hour notice, erasure job
Logs kept ≥ 1 year	Rules 6, 8	Insert-only access log
Access, correction, erasure, grievance, nominee	s.11–14, Rule 14	Rights and grievance module (90-day limit)
Published contact person	s.8(9)	Contact on notice and parent portal

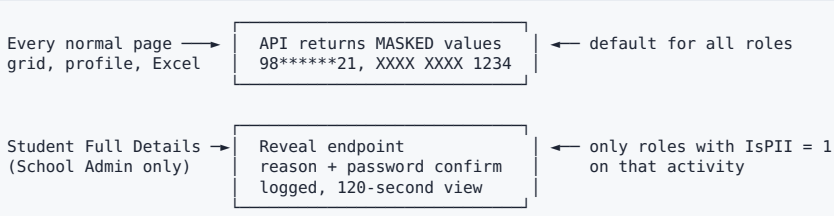
B.4 Penalties (Schedule to the Act)

Failure	Up to
Not taking reasonable security safeguards	₹250 crore
Not reporting a breach to the Board / affected persons	₹200 crore
Breaking children's data obligations	₹200 crore
Other breaches of the Act or Rules	₹50 crore

Security is the biggest risk, which is why masking and encryption (Part C) come first.

Part C — Data protection architecture

C.1 The model in one picture



C.2 Data categories and how each is protected

Category	Examples	Stored as	Normal API	Full Details (after reveal)
IDENTITY	Student / parent name, gender	Plain	Shown	Shown
DOB	Date of birth	Plain	`**/**/2015`	Full
CONTACT	Mobile, alternate mobile	Plain + DDM	`98******21`	Full
CONTACT	Email	Plain + DDM	`ra****@gmail.com`	Full
ADDRESS	Address lines, pincode	Plain + DDM	`H.No ****, Delhi - 110***`	Full
AADHAAR	Student / parent Aadhaar	**Encrypted** + last 4 + HMAC	`XXXX XXXX 1234`	Full
HEALTH	Blood group, medical notes, disability	**Encrypted**	Hidden	Full
FINANCIAL	Bank account, IFSC, family income	**Encrypted**	`XXXXXX4321`	Full
SOCIAL	Caste, religion, category	Plain	By role	Full
PHOTO	Student / parent photo, documents	Files **outside wwwroot**	Thumbnail via secure endpoint	Full image via secure endpoint
LOGIN	Username	Plain	Shown	Shown
LOGIN	**Password**	**Hash only**	Never	**Never — reset only** (see C.7)

C.3 Layer 1 — Encryption of highly sensitive columns

Use application-level **AES-256-GCM** (works with Dapper without driver changes). The database stores only ciphertext; the key comes from configuration (environment variable or Azure Key Vault), never from the database or `appsettings.json` in source control.

For Aadhaar also store:

- `AadhaarLast4 CHAR(4)` for masked display without decrypting.
- `AadhaarHash VARBINARY(32)` (HMAC-SHA256 with a separate key) to find duplicates or search by Aadhaar without decrypting.

> Aadhaar is needed for UDISE+ reporting, so the full number is kept, but only encrypted, in a separate vault table, and never logged. If the school decides it does not need the full number, keep only `AadhaarLast4` and drop the encrypted column.

Vault tables (sensitive columns move out of the main tables)

```
        IF OBJECT_ID('dbo.V3M_DPDP_PiiVault') IS NULL
        CREATE TABLE dbo.V3M_DPDP_PiiVault
        (
            PiiVaultId        BIGINT IDENTITY(1,1) PRIMARY KEY,
            GroupCode         INT            NOT NULL,
            BranchCode        INT            NOT NULL,
            OwnerType         VARCHAR(10)    NOT NULL
                CONSTRAINT CK_DPDP_Vault_Owner CHECK (OwnerType IN ('STUDENT','PARENT','STAFF')),
            OwnerId           INT            NOT NULL,
            AadhaarEnc        VARBINARY(256) NULL,
            AadhaarLast4      CHAR(4)        NULL,
            AadhaarHash       VARBINARY(32)  NULL,
            BloodGroupEnc     VARBINARY(128) NULL,
```

```
    MedicalNotesEnc   VARBINARY(MAX) NULL,
    DisabilityEnc     VARBINARY(512) NULL,
    BankAccountEnc    VARBINARY(256) NULL,
    BankAccountLast4  CHAR(4)        NULL,
    IfscEnc           VARBINARY(128) NULL,
    FamilyIncomeEnc   VARBINARY(128) NULL,
    KeyVersion        TINYINT        NOT NULL DEFAULT 1,
    UpdatedBy         INT            NULL,
    UpdatedOn         DATETIME2(0)   NOT NULL DEFAULT SYSDATETIME(),
    CONSTRAINT UQ_DPDP_Vault_Owner UNIQUE (GroupCode, BranchCode, OwnerType, OwnerId)
);
GO
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = 'IX_DPDP_Vault_AadhaarHash')
    CREATE INDEX IX_DPDP_Vault_AadhaarHash ON dbo.V3M_DPDP_PiiVault (AadhaarHash) WHERE AadhaarHash IS NOT NULL;
GO
```

`IPiiCryptoService`

```
public interface IPiiCryptoService
{
    byte[]? Encrypt(string? plain);
    string? Decrypt(byte[]? cipher);
    byte[]? Hash(string? value);          // HMAC-SHA256 for search / duplicate check
}

public sealed class PiiCryptoService : IPiiCryptoService
{
    private const byte Version = 1;
    private readonly byte[] _encKey;
    private readonly byte[] _hmacKey;

    public PiiCryptoService(IConfiguration cfg)
    {
        _encKey  = Convert.FromBase64String(cfg["Pii:EncryptionKey"] ?? throw new InvalidOperationException("Pii:EncryptionKey miss
        _hmacKey = Convert.FromBase64String(cfg["Pii:HmacKey"]       ?? throw new InvalidOperationException("Pii:HmacKey missing"))
        if (_encKey.Length != 32 || _hmacKey.Length < 32)
            throw new InvalidOperationException("PII keys must be 256-bit.");
    }

    // layout: [version 1][nonce 12][tag 16][ciphertext n]
    public byte[]? Encrypt(string? plain)
    {
        if (string.IsNullOrWhiteSpace(plain)) return null;
        var nonce = RandomNumberGenerator.GetBytes(12);
        var pt    = Encoding.UTF8.GetBytes(plain.Trim());
        var ct    = new byte[pt.Length];
        var tag   = new byte[16];
        using (var aes = new AesGcm(_encKey, 16)) aes.Encrypt(nonce, pt, ct, tag);

        var result = new byte[1 + 12 + 16 + ct.Length];
        result[0] = Version;
        nonce.CopyTo(result, 1);
        tag.CopyTo(result, 13);
        ct.CopyTo(result, 29);
        CryptographicOperations.ZeroMemory(pt);
        return result;
    }

    public string? Decrypt(byte[]? cipher)
    {
        if (cipher is null || cipher.Length < 29 || cipher[0] != Version) return null;
        var pt = new byte[cipher.Length - 29];
        using (var aes = new AesGcm(_encKey, 16))
            aes.Decrypt(cipher.AsSpan(1, 12), cipher.AsSpan(29), cipher.AsSpan(13, 16), pt);
        return Encoding.UTF8.GetString(pt);
    }

    public byte[]? Hash(string? value)
    {
        if (string.IsNullOrWhiteSpace(value)) return null;
        var normalised = new string(value.Where(char.IsLetterOrDigit).ToArray()).ToUpperInvariant();
        return HMACSHA256.HashData(_hmacKey, Encoding.UTF8.GetBytes(normalised));
    }
}

// Program.cs
builder.Services.AddSingleton<IPiiCryptoService, PiiCryptoService>();
```

Generate keys once (PowerShell): `[Convert]::ToBase64String((1..32 | % { Get-Random -Max 256 }))` — better, use `RandomNumberGenerator.GetBytes(32)` in a small console app. Store as environment variables `Pii__EncryptionKey` and `Pii__HmacKey` on the API server. **Back up the keys separately from the database backup** — lost key = lost Aadhaar data.

One-time migration of existing data

1. Create `V3M_DPDP_PiiVault`.
2. Small console job (`SchoolEye.PiiMigration`) reads existing Aadhaar / health / bank columns in batches of 500, encrypts with `PiiCryptoService`, saves to the vault with a TVP (`TT_DPDP_PiiVaultRow`) through `USP_DPDP_SavePiiVaultBatch`.
3. Verify counts match (`COUNT` of non-null source = `COUNT` of vault rows).
4. Update every save/get SP that used the old columns to use the vault instead (Claude Code: list them with `sys.sql_modules`

search first).
5. After one full backup cycle, set the old plain columns to `NULL`, then drop them in a later release.

```
-- find every SP that touches the old Aadhaar / health / bank columns
SELECT OBJECT_NAME(object_id) AS ProcName
FROM sys.sql_modules
WHERE definition LIKE '%Aadhar%' OR definition LIKE '%Aadhaar%'
   OR definition LIKE '%BloodGroup%' OR definition LIKE '%Medical%'
   OR definition LIKE '%AccountNo%' OR definition LIKE '%IFSC%'
ORDER BY ProcName;
```

Database-level protection

- [ ] **TDE** on the database (encrypts data files and backups at rest). Back up the TDE certificate.
- [ ] Connection string: `Encrypt=True;TrustServerCertificate=False`.
- [ ] HTTPS only: `app.UseHsts(); app.UseHttpsRedirection();`, TLS 1.2+.

C.4 Layer 2 — Masking in the API (the main control)

All masking happens **in the API**, so Blazor and every client receive already-masked values. Raw values never leave the server except through the reveal endpoint.

Mask rules

Type	Input	Output
Mobile	`9876543221`	`98******21`
Email	`rahul.sharma@gmail.com`	`ra****@gmail.com`
Aadhaar	`123456781234`	`XXXX XXXX 1234`
Address	`H.No 12, Lajpat Nagar, Delhi 110024`	`H.No ****, ****, Delhi 110***` (keep city)
DOB	`2015-04-12`	`**/**/2015`
BankAccount	`00112233444321`	`XXXXXXXXX4321`
Hidden	any	`""` (health notes, disability, income)

Attribute on model properties

```
public enum PiiType { Mobile, Email, Aadhaar, Address, Dob, BankAccount, Hidden }

[AttributeUsage(AttributeTargets.Property)]
public sealed class PersonalDataAttribute : Attribute
{
    public PiiType Type { get; }
    public PersonalDataAttribute(PiiType type) => Type = type;
}

// example: existing model in DomainModel
public class StudentListModel
{
    public int StudentId { get; set; }
    public string? StudentName { get; set; }
    [PersonalData(PiiType.Mobile)]  public string? MobileNo { get; set; }
    [PersonalData(PiiType.Email)]   public string? EmailId { get; set; }
    [PersonalData(PiiType.Address)] public string? Address { get; set; }
    [PersonalData(PiiType.Dob)]     public string? DOB { get; set; }
}
```

> DOB and other non-string fields: if the model uses `DateTime`, add a string display property for the masked value, or keep the field as `string` in list models. Claude Code: check each model; do not change types the UI depends on without updating the UI.

`IMaskingService`

```
public interface IMaskingService
{
    string Mask(string? value, PiiType type);
    void MaskObject(object? root);     // walks models, lists, ApiResponse<T>
}

public sealed class MaskingService : IMaskingService
{
    private static readonly ConcurrentDictionary<Type, (PropertyInfo Prop, PiiType Type)[]> _piiProps = new();
    private static readonly ConcurrentDictionary<Type, PropertyInfo[]> _childProps = new();

    public string Mask(string? v, PiiType type)
    {
        if (string.IsNullOrWhiteSpace(v)) return v ?? "";
```

```
            v = v.Trim();
            return type switch
            {
                PiiType.Mobile      => v.Length < 4 ? "****" : v[..2] + new string('*', v.Length - 4) + v[^2..],
                PiiType.Email       => MaskEmail(v),
                PiiType.Aadhaar     => v.Length < 4 ? "XXXX" : "XXXX XXXX " + v[^4..],
                PiiType.BankAccount => v.Length < 4 ? "XXXX" : new string('X', v.Length - 4) + v[^4..],
                PiiType.Dob         => v.Length >= 4 ? "**/**/" + ExtractYear(v) : "**/**/****",
                PiiType.Address     => MaskAddress(v),
                PiiType.Hidden      => "",
                _                   => v
            };
        }

        private static string MaskEmail(string v)
        {
            var at = v.IndexOf('@');
            if (at <= 0) return "****";
            var name = v[..at];
            return (name.Length <= 2 ? name[..1] : name[..2]) + "****" + v[at..];
        }

        private static string ExtractYear(string v) =>
            DateTime.TryParse(v, out var d) ? d.Year.ToString() : "****";

        private static string MaskAddress(string v)
        {
            var parts = v.Split(',', StringSplitOptions.TrimEntries);
            if (parts.Length == 1) return "****";
            var last = parts[^1];                                  // "Delhi 110024"
            var masked = System.Text.RegularExpressions.Regex.Replace(last, @"(\d{3})\d{3}", "$1***");
            return string.Join(", ", parts[..^1].Select(_ => "****")) + ", " + masked;
        }

        public void MaskObject(object? root) => Walk(root, new HashSet<object>(ReferenceEqualityComparer.Instance), 0);

        private void Walk(object? node, HashSet<object> seen, int depth)
        {
            if (node is null || depth > 8 || node is string || node.GetType().IsPrimitive || !seen.Add(node)) return;

            if (node is System.Collections.IEnumerable list)
            {
                foreach (var item in list) Walk(item, seen, depth + 1);
                return;
            }

            var t = node.GetType();
            foreach (var (prop, type) in _piiProps.GetOrAdd(t, Scan))
                if (prop.GetValue(node) is string s) prop.SetValue(node, Mask(s, type));

            foreach (var child in _childProps.GetOrAdd(t, ScanChildren))
                Walk(child.GetValue(node), seen, depth + 1);
        }

        private static (PropertyInfo, PiiType)[] Scan(Type t) =>
            t.GetProperties(BindingFlags.Public | BindingFlags.Instance)
             .Where(p => p.CanRead && p.CanWrite && p.PropertyType == typeof(string))
             .Select(p => (p, p.GetCustomAttribute<PersonalDataAttribute>()))
             .Where(x => x.Item2 != null)
             .Select(x => (x.p, x.Item2!.Type)).ToArray();

        // only descend into our own models (DomainModel.*) and collections, plus ApiResponse<T>.Data
        private static PropertyInfo[] ScanChildren(Type t) =>
            t.GetProperties(BindingFlags.Public | BindingFlags.Instance)
             .Where(p => p.CanRead && p.GetIndexParameters().Length == 0 && p.PropertyType != typeof(string)
                      && !p.PropertyType.IsValueType
                      && (p.PropertyType.Namespace?.StartsWith("DomainModel") == true
                          || typeof(System.Collections.IEnumerable).IsAssignableFrom(p.PropertyType)
                          || p.PropertyType == typeof(object)))
             .ToArray();
    }
```

> Claude Code: adjust the `DomainModel` namespace filter to the real model namespaces, and make sure `ApiResponse<T>.Data` is reached.

Global result filter — masks every response automatically

```
    public sealed class PiiMaskingResultFilter : IAsyncResultFilter
    {
        public const string RevealGrantedKey = "Dpdp.RevealGranted";
        private readonly IMaskingService _masker;
        public PiiMaskingResultFilter(IMaskingService masker) => _masker = masker;

        public async Task OnResultExecutionAsync(ResultExecutingContext ctx, ResultExecutionDelegate next)
        {
            var revealGranted = ctx.HttpContext.Items.TryGetValue(RevealGrantedKey, out var v) && v is true;
            if (!revealGranted && ctx.Result is ObjectResult { Value: not null } obj)
                _masker.MaskObject(obj.Value);
            await next();
        }
    }
```

```
// Program.cs
builder.Services.AddSingleton<IMaskingService, MaskingService>();
builder.Services.AddScoped<PiiMaskingResultFilter>();
builder.Services.AddControllers(o => o.Filters.AddService<PiiMaskingResultFilter>());
```

Because the filter is global, **any existing endpoint becomes masked as soon as its model properties get `[PersonalData]`.** No controller code changes.

Where to add `[PersonalData]` (Claude Code task)

1. Run the column-discovery query (Step E2) and the list of models in `DomainModel`.
2. For every model property holding mobile, email, address, DOB, Aadhaar, bank, health, add the attribute.
3. Pages to verify first (they carry `IsPII = 1` in `MST_ACMapping` today): View Student, View Student Ledger, View Student Invoice, Take Student Fee New, Generate Student Invoice, View Invoice Dues, and every Admission / Student / Parent / Staff / Enquiry list.

What still uses raw values (server side only)

SMS, WhatsApp, email and payment services read raw mobile/email **inside the API** from the database. Masking affects only HTTP responses, so sending messages keeps working.

For "call this parent" needs in the office, add a **Click-to-call / Send SMS** button that sends the message from the server, instead of revealing the number.

Excel / PDF exports

Exports are built from the same masked models, so they are masked automatically. Only the Full Details page can show raw data, and it has **no export and no print** (Part D).

C.5 Layer 3 — SQL Server Dynamic Data Masking (defence in depth)

DDM masks plain columns for anyone running ad-hoc queries with a login that lacks `UNMASK` (support staff, report tools, a leaked read-only login).

```
-- example: adjust table/column names after Step E2
ALTER TABLE dbo.StudentMaster ALTER COLUMN MobileNo  ADD MASKED WITH (FUNCTION = 'partial(2,"******",2)');
ALTER TABLE dbo.StudentMaster ALTER COLUMN EmailId   ADD MASKED WITH (FUNCTION = 'email()');
ALTER TABLE dbo.StudentMaster ALTER COLUMN Address   ADD MASKED WITH (FUNCTION = 'default()');
ALTER TABLE dbo.ParentMaster  ALTER COLUMN MobileNo  ADD MASKED WITH (FUNCTION = 'partial(2,"******",2)');
ALTER TABLE dbo.ParentMaster  ALTER COLUMN EmailId   ADD MASKED WITH (FUNCTION = 'email()');

-- the application login must still read raw values (it masks in the API)
GRANT UNMASK TO [SchoolEyeApiUser];
-- support / reporting logins do NOT get UNMASK
```

> DDM is not encryption; a user with `UNMASK` or `db_owner` sees everything. It is an extra layer, not a replacement for API masking.

C.6 Controlled reveal — the only door to raw data

Control	Rule
Who	Roles whose `MST_ACMapping` row for the Student Full Details activity has `IsPII = 1` (School Admin by default)
Server check	Every reveal call re-checks the role in SQL (`V3M_DPDP_CheckPiiPermission`). The Blazor menu flag is not trusted.
Reason	Mandatory dropdown + remarks: Parent request, UDISE/Board submission, TC/Certificate, Emergency contact, Legal/Police request, Other
Re-authentication	Admin re-enters their password (or OTP) once per 15 minutes
Time limit	Revealed values shown for **120 seconds**, then cleared from the page
Rate limit	Max **30 reveals per hour** per user; above that blocked and DPO alerted
Logging	One `V3M_DPDP_AccessLog` row per reveal: who, which student, which fields, reason, IP, time
No copying out	No Excel, no print button, `@media print` hides the panel, screen watermark with admin name + time

C.7 Passwords — never shown, only reset

The request to show students' passwords on the admin page is **not implemented as asked**, on purpose:

- A password that an admin can read is stored either in plain text or with reversible encryption. A breach then exposes every

login, and parents often reuse passwords elsewhere. That is exactly the "reasonable security safeguards" failure that carries the ₹250 crore penalty.
- Showing a password also lets anyone with admin access log in as the parent or student without trace.

What the page does instead:

Shown	Action available
Username	—
Account status (Active / Locked / Disabled)	Unlock / Disable
Last login date and IP	—
Must change password flag	—
—	**Reset Password**: generates a one-time temporary password, shown **once** to the admin, user must change it at next login, logged

Password storage fix (if `MstUsers` holds plain passwords)

Claude Code first checks the password column type/length in `script.sql` and the login SP. If passwords are plain or reversibly encrypted:

```
        IF COL_LENGTH('dbo.MstUsers', 'PasswordHash') IS NULL
            ALTER TABLE dbo.MstUsers ADD PasswordHash NVARCHAR(200) NULL,
                                         MustChangePassword BIT NOT NULL DEFAULT 0,
                                         PasswordChangedOn DATETIME2(0) NULL,
                                         FailedLoginCount INT NOT NULL DEFAULT 0,
                                         LockedUntil DATETIME2(0) NULL;
```

```
        // uses Microsoft.Extensions.Identity.Core (PBKDF2, salted, versioned)
        private static readonly PasswordHasher<object> _hasher = new();

        public string HashPassword(string password) => _hasher.HashPassword(null!, password);

        public bool VerifyPassword(string hash, string password) =>
            _hasher.VerifyHashedPassword(null!, hash, password) != PasswordVerificationResult.Failed;
```

Lazy migration at login:

1. If `PasswordHash` is not null → verify against the hash.
2. Else verify against the old column; if it matches, write `PasswordHash`, clear the old column.
3. After 90 days, force-reset every account still without a hash, then drop the old column.

Part D — Student Full Details page (School Admin)

D.1 What the School Admin sees

Route: `/studentfulldetails` · **Menu:** School Admin → Privacy (DPDP) → *Student Full Details*

Flow

```
Session (PageHeading) ─► sliders popup: Class, Section, Student No ─► Search
        │
        ▼
Masked result grid (Student No, Name, Class-Section, Roll, Father, Mobile masked)
        │ click a row
        ▼
Full profile — MASKED by default
  [ Student ] [ Parents ] [ Address ] [ Health & Bank ] [ Login ] [ Documents ]
        │ click "Reveal full details"
        ▼
Reason + remarks + confirm password ─► unmasked for 120 s ─► auto-hide
```

Sections on the profile

Section	Fields
Student	Photo, Student No, name, gender, DOB, class, section, roll no, session, admission date, category, religion, mobile, email, Aadhaar
Parents	Father / mother / guardian: name, photo, mobile, alternate mobile, email, occupation, qualification, Aadhaar
Address	Current and permanent address, city, state, pincode
Health & Bank	Blood group, medical notes, disability, bank account, IFSC, family income
Login	Student and parent username, status, last login, Reset Password button
Documents	List of uploaded documents; open through the secure file endpoint

D.2 Register the page in the menu and permission tables

Uses the existing `MSTModule` → `MstFeaturesList` → `MstActivityList` → `MST_ACMapping` structure and the existing `IsPII` flag.

```
-- Claude Code: confirm exact column lists of MstFeaturesList / MstActivityList in script.sql
DECLARE @ModuleId INT = 1006;   -- School Admin (confirm)
DECLARE @FeatureId INT, @ActivityId INT;

IF NOT EXISTS (SELECT 1 FROM MstFeaturesList WHERE ModuleId = @ModuleId AND FeaturesName = 'Privacy (DPDP)')
BEGIN
    INSERT INTO MstFeaturesList (ModuleId, FeaturesName, Description, DisplayOrder, CreatedBy, CreatedDate, IsValid, Icon)
    VALUES (@ModuleId, 'Privacy (DPDP)', 'DPDP compliance', 900, 'system', GETDATE(), 1, 'bi bi-shield-lock-fill');
END
SELECT @FeatureId = FeatureId FROM MstFeaturesList WHERE ModuleId = @ModuleId AND FeaturesName = 'Privacy (DPDP)';

IF NOT EXISTS (SELECT 1 FROM MstActivityList WHERE URL = '/studentfulldetails')
BEGIN
    INSERT INTO MstActivityList (FeatureId, ActivityName, URL, DisplayOrder, DisplayName, LabelIcon /*, other required cols */)
    VALUES (@FeatureId, 'Student Full Details', '/studentfulldetails', 10, 'Student Full Details', 'bi bi-person-vcard-fill');
END
SELECT @ActivityId = ActivityId FROM MstActivityList WHERE URL = '/studentfulldetails';

-- give School Admin role (RoleId to confirm) access with IsPII = 1
INSERT INTO MST_ACMapping (RoleId, ModuleId, FeatureId, ActivityId,
                           IsAdd, IsModifiy, IsPrint, IsExportToExcel, IsPII, Action1, Action2, Action3)
SELECT r.RoleId, @ModuleId, @FeatureId, @ActivityId, 0, 1, 0, 0, 1, 0, 0, 0
FROM MstRoles r
WHERE r.RoleName = 'School Admin'
  AND NOT EXISTS (SELECT 1 FROM MST_ACMapping m WHERE m.RoleId = r.RoleId AND m.ActivityId = @ActivityId);
```

`IsModifiy = 1` here means "can reset password / unlock". `IsPrint` and `IsExportToExcel` stay 0.

D.3 Stored procedures

Permission check (used by the reveal guard)

```
CREATE OR ALTER PROCEDURE dbo.V3M_DPDP_CheckPiiPermission
    @RoleId INT,
    @URL    VARCHAR(200)
AS
BEGIN
    SET NOCOUNT ON;
    SELECT CAST(ISNULL(MAX(CAST(AM.IsPII AS INT)), 0) AS BIT) AS IsPII,
           CAST(ISNULL(MAX(CAST(AM.IsModifiy AS INT)), 0) AS BIT) AS IsModify
```

```
    FROM MST_ACMapping AM WITH (NOLOCK)
    INNER JOIN MstActivityList MA WITH (NOLOCK) ON MA.ActivityId = AM.ActivityId
    INNER JOIN MstFeaturesList MF WITH (NOLOCK) ON MF.FeatureId  = AM.FeatureId AND MF.IsValid = 1
    INNER JOIN MSTModule       MM WITH (NOLOCK) ON MM.ModuleId   = AM.ModuleId  AND MM.IsValid = 1
    WHERE AM.RoleId = @RoleId
      AND MA.URL = @URL;
END
```

Search (list, masked by the API)

```
CREATE OR ALTER PROCEDURE dbo.V3M_DPDP_StudentFullDetails_Search
    @GroupCode  INT,
    @BranchCode INT,
    @SessionId  INT,
    @ClassId    INT = NULL,
    @SectionId  INT = NULL,
    @StudentNo  VARCHAR(30) = NULL
AS
BEGIN
    SET NOCOUNT ON;
    SELECT TOP (200)
        S.StudentId,
        S.StudentNo,
        S.StudentName,
        C.ClassName,
        SC.SectionName,
        SM.RollNo,
        P.FatherName,
        S.MobileNo
    FROM dbo.StudentMaster S WITH (NOLOCK)
    INNER JOIN dbo.StudentSessionMap SM WITH (NOLOCK) ON SM.StudentId = S.StudentId AND SM.SessionId = @SessionId
    INNER JOIN dbo.ClassMaster   C  WITH (NOLOCK) ON C.ClassId    = SM.ClassId
    LEFT  JOIN dbo.SectionMaster SC WITH (NOLOCK) ON SC.SectionId = SM.SectionId
    LEFT  JOIN dbo.ParentMaster  P  WITH (NOLOCK) ON P.StudentId  = S.StudentId
    WHERE S.GroupCode  = @GroupCode
      AND S.BranchCode = @BranchCode
      AND (@ClassId   IS NULL OR SM.ClassId   = @ClassId)
      AND (@SectionId IS NULL OR SM.SectionId = @SectionId)
      AND (@StudentNo IS NULL OR S.StudentNo  = @StudentNo)
    ORDER BY C.ClassName, SC.SectionName, SM.RollNo;
END
```

> Placeholders (`StudentMaster`, `ParentMaster`, column names) are mapped in Step E1. `TOP (200)` prevents bulk harvesting through this page.

Full profile (three result sets)

```
CREATE OR ALTER PROCEDURE dbo.V3M_DPDP_StudentFullDetails_Get
    @GroupCode  INT,
    @BranchCode INT,
    @SessionId  INT,
    @StudentId  INT
AS
BEGIN
    SET NOCOUNT ON;

    -- 1. Student
    SELECT S.StudentId, S.StudentNo, S.StudentName, S.Gender,
           CONVERT(VARCHAR(10), S.DOB, 23) AS DOB,
           C.ClassName, SC.SectionName, SM.RollNo, S.AdmissionDate,
           S.Category, S.Religion, S.MobileNo, S.EmailId,
           S.CurrentAddress, S.PermanentAddress, S.City, S.State, S.Pincode,
           S.PhotoPath,
           V.AadhaarLast4, V.AadhaarEnc, V.BloodGroupEnc, V.MedicalNotesEnc,
           V.DisabilityEnc, V.BankAccountEnc, V.BankAccountLast4, V.IfscEnc, V.FamilyIncomeEnc
    FROM dbo.StudentMaster S WITH (NOLOCK)
    INNER JOIN dbo.StudentSessionMap SM WITH (NOLOCK) ON SM.StudentId = S.StudentId AND SM.SessionId = @SessionId
    INNER JOIN dbo.ClassMaster   C  WITH (NOLOCK) ON C.ClassId    = SM.ClassId
    LEFT  JOIN dbo.SectionMaster SC WITH (NOLOCK) ON SC.SectionId = SM.SectionId
    LEFT  JOIN dbo.V3M_DPDP_PiiVault V WITH (NOLOCK)
           ON V.OwnerType = 'STUDENT' AND V.OwnerId = S.StudentId
          AND V.GroupCode = S.GroupCode AND V.BranchCode = S.BranchCode
    WHERE S.StudentId = @StudentId AND S.GroupCode = @GroupCode AND S.BranchCode = @BranchCode;

    -- 2. Parents / guardian
    SELECT P.ParentId, P.Relation, P.ParentName, P.MobileNo, P.AlternateMobileNo, P.EmailId,
           P.Occupation, P.Qualification, P.PhotoPath,
           V.AadhaarLast4, V.AadhaarEnc
    FROM dbo.ParentMaster P WITH (NOLOCK)
    LEFT JOIN dbo.V3M_DPDP_PiiVault V WITH (NOLOCK)
           ON V.OwnerType = 'PARENT' AND V.OwnerId = P.ParentId
          AND V.GroupCode = @GroupCode AND V.BranchCode = @BranchCode
    WHERE P.StudentId = @StudentId;

    -- 3. Logins (never the password)
    SELECT U.UserSid, U.UserName,
           CASE WHEN U.LockedUntil > SYSDATETIME() THEN 'LOCKED'
                WHEN U.IsValid = 0 THEN 'DISABLED' ELSE 'ACTIVE' END AS AccountStatus,
           U.LastLoginOn, U.LastLoginIp, U.MustChangePassword,
```

```
               CASE WHEN U.StudentId IS NOT NULL THEN 'STUDENT' ELSE 'PARENT' END AS LoginType
    FROM dbo.MstUsers U WITH (NOLOCK)
    WHERE U.StudentId = @StudentId
       OR U.ParentId IN (SELECT ParentId FROM dbo.ParentMaster WHERE StudentId = @StudentId);
END
```

> The login result set lists only safe columns. `SELECT *` from `MstUsers` is forbidden anywhere in the project.

Reveal log (extends the access log)

```
CREATE OR ALTER PROCEDURE dbo.USP_DPDP_InsertRevealLog
    @GroupCode    INT,
    @BranchCode   INT,
    @LoginId      INT,
    @StudentId    INT,
    @Fields       VARCHAR(500),
    @ReasonCode   VARCHAR(30),
    @ReasonText   NVARCHAR(300),
    @IpAddress    VARCHAR(45),
    @RevealsInLastHour INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;

    SELECT @RevealsInLastHour = COUNT(1)
    FROM dbo.V3M_DPDP_AccessLog WITH (NOLOCK)
    WHERE LoginId = @LoginId AND Action = 'UNMASK'
      AND LoggedOn >= DATEADD(HOUR, -1, SYSDATETIME());

    INSERT INTO dbo.V3M_DPDP_AccessLog
        (GroupCode, BranchCode, LoginId, Action, Entity, EntityId, Fields, Endpoint, IpAddress, ReasonCode, ReasonText)
    VALUES
        (@GroupCode, @BranchCode, @LoginId, 'UNMASK', 'STUDENT', CAST(@StudentId AS VARCHAR(50)),
         @Fields, '/api/Dpdp/StudentFullDetails/Reveal', @IpAddress, @ReasonCode, @ReasonText);
END
```

Reset password

```
CREATE OR ALTER PROCEDURE dbo.USP_DPDP_ResetUserPassword
    @UserSid      INT,
    @PasswordHash NVARCHAR(200),
    @ResetBy      INT,
    @Message      NVARCHAR(200) OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    UPDATE dbo.MstUsers
    SET PasswordHash = @PasswordHash,
        MustChangePassword = 1,
        PasswordChangedOn = SYSDATETIME(),
        FailedLoginCount = 0,
        LockedUntil = NULL
    WHERE UserSid = @UserSid;

    SET @Message = CASE WHEN @@ROWCOUNT = 1 THEN N'Password reset.' ELSE N'User not found.' END;
END
```

The API also writes an access-log row with `Action = 'RESET_PWD'`.

D.4 Models

```
public class StudentFullDetailsSearchModel
{
    public int StudentId { get; set; }
    public string? StudentNo { get; set; }
    public string? StudentName { get; set; }
    public string? ClassName { get; set; }
    public string? SectionName { get; set; }
    public string? RollNo { get; set; }
    public string? FatherName { get; set; }
    [PersonalData(PiiType.Mobile)] public string? MobileNo { get; set; }
}

public class StudentFullDetailsModel
{
    public StudentProfileModel Student { get; set; } = new();
    public List<ParentProfileModel> Parents { get; set; } = new();
    public List<StudentLoginModel> Logins { get; set; } = new();
    public bool IsRevealed { get; set; }
    public DateTime? RevealExpiresOn { get; set; }
}

public class StudentProfileModel
{
    public int StudentId { get; set; }
    public string? StudentNo { get; set; }
```

```
        public string? StudentName { get; set; }
        public string? Gender { get; set; }
        [PersonalData(PiiType.Dob)]      public string? DOB { get; set; }
        public string? ClassName { get; set; }
        public string? SectionName { get; set; }
        public string? RollNo { get; set; }
        public DateTime? AdmissionDate { get; set; }
        public string? Category { get; set; }
        public string? Religion { get; set; }
        [PersonalData(PiiType.Mobile)]   public string? MobileNo { get; set; }
        [PersonalData(PiiType.Email)]    public string? EmailId { get; set; }
        [PersonalData(PiiType.Address)]  public string? CurrentAddress { get; set; }
        [PersonalData(PiiType.Address)]  public string? PermanentAddress { get; set; }
        public string? City { get; set; }
        public string? State { get; set; }
        public string? Pincode { get; set; }
        public bool HasPhoto { get; set; }                      // path never sent to the client
        [PersonalData(PiiType.Aadhaar)]  public string? Aadhaar { get; set; }
        [PersonalData(PiiType.Hidden)]   public string? BloodGroup { get; set; }
        [PersonalData(PiiType.Hidden)]   public string? MedicalNotes { get; set; }
        [PersonalData(PiiType.Hidden)]   public string? Disability { get; set; }
        [PersonalData(PiiType.BankAccount)] public string? BankAccount { get; set; }
        [PersonalData(PiiType.Hidden)]   public string? Ifsc { get; set; }
        [PersonalData(PiiType.Hidden)]   public string? FamilyIncome { get; set; }
    }

    public class ParentProfileModel
    {
        public int ParentId { get; set; }
        public string? Relation { get; set; }
        public string? ParentName { get; set; }
        [PersonalData(PiiType.Mobile)]  public string? MobileNo { get; set; }
        [PersonalData(PiiType.Mobile)]  public string? AlternateMobileNo { get; set; }
        [PersonalData(PiiType.Email)]   public string? EmailId { get; set; }
        public string? Occupation { get; set; }
        public string? Qualification { get; set; }
        public bool HasPhoto { get; set; }
        [PersonalData(PiiType.Aadhaar)] public string? Aadhaar { get; set; }
    }

    public class StudentLoginModel
    {
        public int UserSid { get; set; }
        public string? UserName { get; set; }
        public string? AccountStatus { get; set; }
        public DateTime? LastLoginOn { get; set; }
        public string? LastLoginIp { get; set; }
        public bool MustChangePassword { get; set; }
        public string? LoginType { get; set; }
    }

    public class RevealRequestModel
    {
        public int SessionId { get; set; }
        public int StudentId { get; set; }
        public string ReasonCode { get; set; } = "";     // PARENT_REQ, UDISE, TC_CERT, EMERGENCY, LEGAL, OTHER
        public string? ReasonText { get; set; }
        public string ConfirmPassword { get; set; } = "";
    }
```

In the masked path, `Aadhaar` and `BankAccount` are filled from the `Last4` columns as `"XXXXXXXX" + last4`, so the masker shows `XXXX XXXX 1234` without ever decrypting. The encrypted value is decrypted **only** in the reveal path.

D.5 Service (Dapper)

```
    public interface IStudentFullDetailsService
    {
        Task<List<StudentFullDetailsSearchModel>> SearchAsync(int groupCode, int branchCode, int sessionId, int? classId, int? sectionI
string? studentNo);
        Task<StudentFullDetailsModel?> GetAsync(int groupCode, int branchCode, int sessionId, int studentId, bool decrypt);
        Task<(bool IsPII, bool IsModify)> CheckPermissionAsync(int roleId, string url);
        Task<int> InsertRevealLogAsync(int groupCode, int branchCode, int loginId, int studentId, string fields, string reasonCode, str
reasonText, string? ip);
        Task<string> ResetPasswordAsync(int userSid, string passwordHash, int resetBy);
        Task<string?> GetPhotoPathAsync(int groupCode, int branchCode, string ownerType, int ownerId);
    }

    public class StudentFullDetailsService : IStudentFullDetailsService
    {
        private readonly IDbConnectionFactory _db;        // use the existing connection factory
        private readonly IPiiCryptoService _crypto;

        public StudentFullDetailsService(IDbConnectionFactory db, IPiiCryptoService crypto)
        { _db = db; _crypto = crypto; }

        public async Task<List<StudentFullDetailsSearchModel>> SearchAsync(int groupCode, int branchCode, int sessionId, int? classId,
sectionId, string? studentNo)
        {
            using var conn = _db.CreateConnection();
            var param = new DynamicParameters();
```

```
            param.Add("@GroupCode", groupCode);
            param.Add("@BranchCode", branchCode);
            param.Add("@SessionId", sessionId);
            param.Add("@ClassId", classId);
            param.Add("@SectionId", sectionId);
            param.Add("@StudentNo", string.IsNullOrWhiteSpace(studentNo) ? null : studentNo.Trim());
            var rows = await conn.QueryAsync<StudentFullDetailsSearchModel>(
                "dbo.V3M_DPDP_StudentFullDetails_Search", param, commandType: CommandType.StoredProcedure);
            return rows.ToList();
        }

        public async Task<StudentFullDetailsModel?> GetAsync(int groupCode, int branchCode, int sessionId, int studentId, bool decrypt)
        {
            using var conn = _db.CreateConnection();
            var param = new DynamicParameters();
            param.Add("@GroupCode", groupCode);
            param.Add("@BranchCode", branchCode);
            param.Add("@SessionId", sessionId);
            param.Add("@StudentId", studentId);

            using var grid = await conn.QueryMultipleAsync(
                "dbo.V3M_DPDP_StudentFullDetails_Get", param, commandType: CommandType.StoredProcedure);

            var s = await grid.ReadFirstOrDefaultAsync<StudentRow>();
            if (s is null) return null;
            var parents = (await grid.ReadAsync<ParentRow>()).ToList();
            var logins  = (await grid.ReadAsync<StudentLoginModel>()).ToList();

            return new StudentFullDetailsModel
            {
                Student = new StudentProfileModel
                {
                    StudentId = s.StudentId, StudentNo = s.StudentNo, StudentName = s.StudentName, Gender = s.Gender,
                    DOB = s.DOB, ClassName = s.ClassName, SectionName = s.SectionName, RollNo = s.RollNo,
                    AdmissionDate = s.AdmissionDate, Category = s.Category, Religion = s.Religion,
                    MobileNo = s.MobileNo, EmailId = s.EmailId,
                    CurrentAddress = s.CurrentAddress, PermanentAddress = s.PermanentAddress,
                    City = s.City, State = s.State, Pincode = s.Pincode,
                    HasPhoto = !string.IsNullOrEmpty(s.PhotoPath),
                    Aadhaar      = decrypt ? _crypto.Decrypt(s.AadhaarEnc) : Last4(s.AadhaarLast4),
                    BloodGroup   = decrypt ? _crypto.Decrypt(s.BloodGroupEnc) : null,
                    MedicalNotes = decrypt ? _crypto.Decrypt(s.MedicalNotesEnc) : null,
                    Disability   = decrypt ? _crypto.Decrypt(s.DisabilityEnc) : null,
                    BankAccount  = decrypt ? _crypto.Decrypt(s.BankAccountEnc) : Last4(s.BankAccountLast4),
                    Ifsc         = decrypt ? _crypto.Decrypt(s.IfscEnc) : null,
                    FamilyIncome = decrypt ? _crypto.Decrypt(s.FamilyIncomeEnc) : null
                },
                Parents = parents.Select(p => new ParentProfileModel
                {
                    ParentId = p.ParentId, Relation = p.Relation, ParentName = p.ParentName,
                    MobileNo = p.MobileNo, AlternateMobileNo = p.AlternateMobileNo, EmailId = p.EmailId,
                    Occupation = p.Occupation, Qualification = p.Qualification,
                    HasPhoto = !string.IsNullOrEmpty(p.PhotoPath),
                    Aadhaar = decrypt ? _crypto.Decrypt(p.AadhaarEnc) : Last4(p.AadhaarLast4)
                }).ToList(),
                Logins = logins
            };
        }

        // masked path: give the masker a value whose last 4 digits are real
        private static string? Last4(string? last4) => string.IsNullOrEmpty(last4) ? null : "XXXXXXXX" + last4;

        // CheckPermissionAsync, InsertRevealLogAsync (OUTPUT param with dbType/size),
        // ResetPasswordAsync (OUTPUT @Message), GetPhotoPathAsync: same Dapper pattern.
    }
```

`StudentRow` and `ParentRow` are private row classes matching the SP result columns (including the `byte[]` encrypted fields). They are never returned from the API.

D.6 Controller

```
    [Authorize]
    [ApiController]
    [Route("api/Dpdp/StudentFullDetails")]
    public class StudentFullDetailsController : ControllerBase
    {
        private const string PageUrl = "/studentfulldetails";
        private static readonly HashSet<string> Reasons = new() { "PARENT_REQ", "UDISE", "TC_CERT", "EMERGENCY", "LEGAL", "OTHER" };

        private readonly IStudentFullDetailsService _svc;
        private readonly IUserAuthService _auth;          // existing service that verifies a user's password
        private readonly PasswordHasher<object> _hasher = new();

        public StudentFullDetailsController(IStudentFullDetailsService svc, IUserAuthService auth)
        { _svc = svc; _auth = auth; }

        private int Claim(string name) => int.TryParse(User.FindFirst(name)?.Value, out var v) ? v : 0;

        [HttpGet("Search")]
        [AuditPersonalData("STUDENT", "VIEW")]
        public async Task<IActionResult> Search(int sessionId, int? classId, int? sectionId, string? studentNo)
        {
            var (isPII, _) = await _svc.CheckPermissionAsync(Claim("RoleId"), PageUrl);
```

```
            if (!isPII) return StatusCode(403, ApiResponse<object>.Fail("Not allowed."));

            var data = await _svc.SearchAsync(Claim("GroupCode"), Claim("BranchCode"), sessionId, classId, sectionId, studentNo);
            return Ok(ApiResponse<List<StudentFullDetailsSearchModel>>.Success(data));      // masked by global filter
        }

        [HttpGet("{studentId:int}")]
        [AuditPersonalData("STUDENT", "VIEW")]
        public async Task<IActionResult> Get(int studentId, int sessionId)
        {
            var (isPII, _) = await _svc.CheckPermissionAsync(Claim("RoleId"), PageUrl);
            if (!isPII) return StatusCode(403, ApiResponse<object>.Fail("Not allowed."));

            var data = await _svc.GetAsync(Claim("GroupCode"), Claim("BranchCode"), sessionId, studentId, decrypt: false);
            return data is null ? NotFound() : Ok(ApiResponse<StudentFullDetailsModel>.Success(data));
        }

        [HttpPost("Reveal")]
        public async Task<IActionResult> Reveal([FromBody] RevealRequestModel req)
        {
            var loginId = Claim("LoginId");
            var (isPII, _) = await _svc.CheckPermissionAsync(Claim("RoleId"), PageUrl);
            if (!isPII) return StatusCode(403, ApiResponse<object>.Fail("Not allowed."));

            if (!Reasons.Contains(req.ReasonCode) || (req.ReasonCode == "OTHER" && string.IsNullOrWhiteSpace(req.ReasonText)))
                return BadRequest(ApiResponse<object>.Fail("Select a reason."));

            if (!await _auth.VerifyCurrentUserPasswordAsync(loginId, req.ConfirmPassword))
                return BadRequest(ApiResponse<object>.Fail("Password is incorrect."));

            var count = await _svc.InsertRevealLogAsync(Claim("GroupCode"), Claim("BranchCode"), loginId, req.StudentId,
                "ALL", req.ReasonCode, req.ReasonText, HttpContext.Connection.RemoteIpAddress?.ToString());
            if (count >= 30)
                return StatusCode(429, ApiResponse<object>.Fail("Reveal limit reached. DPO has been notified."));   // + alert DPO

            var data = await _svc.GetAsync(Claim("GroupCode"), Claim("BranchCode"), req.SessionId, req.StudentId, decrypt: true);
            if (data is null) return NotFound();

            data.IsRevealed = true;
            data.RevealExpiresOn = DateTime.Now.AddSeconds(120);
            HttpContext.Items[PiiMaskingResultFilter.RevealGrantedKey] = true;     // skip masking for THIS response only
            Response.Headers.CacheControl = "no-store";
            return Ok(ApiResponse<StudentFullDetailsModel>.Success(data));
        }

        [HttpGet("Photo/{ownerType}/{ownerId:int}")]
        [AuditPersonalData("STUDENT", "VIEW")]
        public async Task<IActionResult> Photo(string ownerType, int ownerId)
        {
            var (isPII, _) = await _svc.CheckPermissionAsync(Claim("RoleId"), PageUrl);
            if (!isPII) return Forbid();
            var path = await _svc.GetPhotoPathAsync(Claim("GroupCode"), Claim("BranchCode"), ownerType, ownerId);
            if (path is null || !System.IO.File.Exists(path)) return NotFound();
            Response.Headers.CacheControl = "no-store";
            return PhysicalFile(path, "image/jpeg");
        }

        [HttpPost("ResetPassword/{userSid:int}")]
        public async Task<IActionResult> ResetPassword(int userSid)
        {
            var (isPII, isModify) = await _svc.CheckPermissionAsync(Claim("RoleId"), PageUrl);
            if (!isPII || !isModify) return StatusCode(403, ApiResponse<object>.Fail("Not allowed."));

            var temp = TempPassword.Generate(10);                  // letters + digits + symbol
            var msg = await _svc.ResetPasswordAsync(userSid, _hasher.HashPassword(null!, temp), Claim("LoginId"));
            // log Action = 'RESET_PWD' (no password in the log)
            HttpContext.Items[PiiMaskingResultFilter.RevealGrantedKey] = true;
            Response.Headers.CacheControl = "no-store";
            return Ok(ApiResponse<object>.Success(new { Message = msg, TemporaryPassword = temp }));   // shown once
        }
    }
```

> Claude Code: match `ApiResponse<T>` factory methods, claim names and the existing password-verify service to the real project. The photo path must resolve to a folder **outside wwwroot**; validate `ownerType` ∈ {STUDENT, PARENT}.

D.7 Blazor page

File: `Components/Pages/Dpdp/StudentFullDetails.razor` · `@page "/studentfulldetails"` · `@rendermode InteractiveServer`

Layout

```
┌ PageHeading ──────────────────────────────────────────────────────┐
│ Student Full Details          [Session ▼]  [≡ sliders]  [Search]  │
└──────────────────────────────────────────────────────────────────┘

  sliders popup (col-xl-9):  [Class ▼]  [Section ▼]  [Student No ____]

┌ QuickGrid (masked) ──────────────────────────────────────────────────────┐
│ Student No │ Name │ Class-Sec │ Roll │ Father │ Mobile      │ [View]     │
│ 2024/0153  │ ...  │ VIII-A    │ 12   │ ...    │ 98******21  │            │
```

```
└──────────────────────────────────────────────────────────────────┘

┌ Profile ──────────────────────────────── [🔓 Reveal full details] ──┐
│ [photo]  Name · Student No · Class-Sec · Roll                      │
│ Tabs: Student | Parents | Address | Health & Bank | Login | Documents    │
│ Masked values ............                                         │
│ (after reveal: values shown, "Hides in 1:47", [Hide now])          │
│ watermark: "Viewed by <admin> · 22-10-2026 10:42" across the panel │
└──────────────────────────────────────────────────────────────────┘
```

Behaviour rules

1. Session dropdown loads the current session by default (existing session API).
2. Class → Section cascade from existing masters.
3. **Search** calls `api/Dpdp/StudentFullDetails/Search`; grid with `Paginator`.
4. **View** calls `GET {studentId}` → masked profile.
5. **Reveal full details** opens a modal: reason dropdown, remarks (required for *Other*), confirm password. On success, replaces the model with the revealed one.
6. A `System.Threading.Timer` (or `PeriodicTimer`) counts down from `RevealExpiresOn`; at zero it calls the masked `GET` again and replaces the model, then clears the revealed object (`revealed = null`).
7. **Hide now**, navigating away, and `Dispose()` all clear the revealed object immediately.
8. Photo: `<img src="api/Dpdp/StudentFullDetails/Photo/STUDENT/@id">` loaded through an authenticated call (convert to a base64 data URL in the component using `HttpClient` with the token, since `<img>` cannot send the JWT).
9. **Reset Password** (Login tab): SweetAlert confirm → call API → show the temporary password once in a SweetAlert with a copy button and the note "User must change it at next login". Never store it in component state after the alert closes.
10. No `AdmitChildButton` export, no print button on this page.
11. CSS: `@media print { .pii-panel { display: none !important; } }` and `user-select: none` on revealed values (deterrent only).
12. Implement `IDisposable`; dispose the timer.

Component skeleton

```
@page "/studentfulldetails"
@rendermode InteractiveServer
@implements IDisposable
@inject Shared.IHttpService httpService
@inject SweetAlertService Alert

<PageHeading Title="Student Full Details">
    <select class="form-select" @bind="sessionId">
        @foreach (var s in sessions) { <option value="@s.SessionId">@s.SessionName</option> }
    </select>
    <button class="btn btn-light" @onclick="() => showFilter = !showFilter"><i class="bi bi-sliders"></i></button>
    <button class="btn btn-primary" @onclick="SearchAsync">Search</button>
</PageHeading>

@if (showFilter)
{
    <div class="col-xl-9 search-popup">
        @* Class, Section, Student No inputs *@
    </div>
}

@* QuickGrid of results with View button *@

@if (profile is not null)
{
    <div class="pii-panel position-relative">
        <div class="pii-watermark">Viewed by @userName · @DateTime.Now:dd-MM-yyyy HH:mm</div>
        @if (!profile.IsRevealed)
        {
            <button class="btn btn-outline-danger" @onclick="OpenRevealModal"><i class="bi bi-unlock"></i> Reveal full details</button>
        }
        else
        {
            <span class="badge bg-danger">Hides in @remaining</span>
            <button class="btn btn-sm btn-secondary" @onclick="HideNowAsync">Hide now</button>
        }
        @* tabs: Student / Parents / Address / Health & Bank / Login / Documents *@
    </div>
}

@code {
    private int sessionId;
    private bool showFilter;
    private StudentFullDetailsModel? profile;
    private PeriodicTimer? timer;
    private string remaining = "";
    private string userName = "";
    // ... SearchAsync, ViewAsync, RevealAsync (POST Reveal), HideNowAsync, countdown loop

    private async Task HideNowAsync()
    {
        timer?.Dispose();
        if (profile is not null)
            profile = await httpService.Get<StudentFullDetailsModel>(
                $"api/Dpdp/StudentFullDetails/{profile.Student.StudentId}?sessionId={sessionId}");
    }
```

```
    public void Dispose() { timer?.Dispose(); profile = null; }
}
```

> Claude Code: build the full page from this skeleton using the existing `PageHeading`, search-popup, QuickGrid and SweetAlert patterns from another SchoolEye page. Copy the structure of an existing list page rather than inventing new CSS.

Part E — Full DPDP module, step by step

Step E1 — Inspect the existing SchoolEye project

Goal: understand how SchoolEye is built before writing DPDP code.

- [ ] List projects: API, ApplicationInterface, Infrastructure, DomainModel, Blazor UI.
- [ ] Read `Program.cs`, `appsettings.json`: auth (JWT), CORS, HTTPS, logging (Serilog?), Swagger, connection string (`Encrypt=True`?).
- [ ] Find JWT claim names for RoleId, LoginId, GroupCode, BranchCode.
- [ ] Open one complete existing API (Model, Interface, Service, Controller, SP); copy its structure.
- [ ] Find the Dapper connection factory and `ApiResponse<T>` helpers.
- [ ] Real types of `GroupCode`, `BranchCode`, `SessionId`, `StudentId`, `ParentId`, `LoginId` in `script.sql`.
- [ ] Fill the placeholder table in A.2.
- [ ] How passwords are stored in `MstUsers` and verified in the login SP.
- [ ] Where photos and documents are stored (wwwroot? folder? DB?).
- [ ] Every SMS / WhatsApp / email sending point.
- [ ] Parent login / portal and how parents authenticate.

Deliverable: `docs/DPDP_DISCOVERY.md` — one page with all of the above. **Wait for approval before Step E2.**

Step E2 — Data inventory

```
SELECT t.name AS TableName, c.name AS ColumnName, ty.name AS DataType, c.max_length
FROM sys.columns c
JOIN sys.tables t  ON t.object_id = c.object_id
JOIN sys.types  ty ON ty.user_type_id = c.user_type_id
WHERE c.name LIKE '%Name%'    OR c.name LIKE '%Mobile%'  OR c.name LIKE '%Phone%'
   OR c.name LIKE '%Email%'   OR c.name LIKE '%Aadha%'   OR c.name LIKE '%DOB%'
   OR c.name LIKE '%Birth%'   OR c.name LIKE '%Address%' OR c.name LIKE '%Photo%'
   OR c.name LIKE '%Blood%'   OR c.name LIKE '%Medical%' OR c.name LIKE '%Health%'
   OR c.name LIKE '%Caste%'   OR c.name LIKE '%Religion%' OR c.name LIKE '%Income%'
   OR c.name LIKE '%Account%' OR c.name LIKE '%IFSC%'    OR c.name LIKE '%PAN%'
   OR c.name LIKE '%Password%' OR c.name LIKE '%Pwd%'
ORDER BY t.name, c.name;
```

Deliverable: seed rows for `V3M_DPDP_DataCategory` (table, column, category, mask rule, encrypt yes/no) + list of models needing `[PersonalData]`.

Step E3 — Governance decisions (school + V3M)

- [ ] **Contact person / DPO** per school (name, email, phone) — shown on every notice.
- [ ] **Data Processing Agreement** school ↔ V3M: purposes, security, breach reporting to the school without delay, deletion at contract end, audit rights.
- [ ] Purposes and legal basis (table below).
- [ ] Retention periods (confirm with counsel).
- [ ] Third parties: SMS gateway, payment gateway, GPS, hosting — sign agreements.
- [ ] Breach team: who decides, who reports to the Board, who informs parents.
- [ ] Which roles get `IsPII = 1` on Student Full Details (default: School Admin only).

Purpose	Legal basis	Consent needed?
Admission, academics, attendance, exams	Exempt (educational activities)	No (notice still required)
Fee collection and receipts	Legitimate use	No
Transport live GPS	Fourth Schedule (child safety)	No
Fee / attendance SMS to parent	Legitimate use	No
UDISE+ / Board submissions (Aadhaar)	Legal obligation	No
Promotional SMS / WhatsApp	Consent	**Yes**
Photos on website / social media	Consent	**Yes**
Alumni database after TC	Consent	**Yes**
Sharing with third-party apps / coaching	Consent	**Yes**

Data	Trigger	Suggested retention
Enquiry not converted	Enquiry closed	1 year, then erase
Admission / TC register	—	Permanent (state rules)
Fee and accounts	Financial year end	8 years, then anonymise
Website photos	Consent withdrawn or TC	Erase within 30 days
Student contact, health, Aadhaar	TC issued	1 year, then erase
Staff records	Relieved	As per labour / PF law
Access logs	Logged	Minimum 1 year

Step E4 — Create the DPDP database objects

- [ ] Run Appendix A on a test DB after replacing `INT` placeholders with real types; then production after a backup.
- [ ] Uncomment the `DENY UPDATE, DELETE` on the access log for the API login.
- [ ] Seed purposes, data categories, retention policies from E2–E3.

Step E5 — Security foundation (Part C)

Implement in this order, one PR each:

1. `IPiiCryptoService` + keys in environment + vault table + migration job.
2. `IMaskingService` + `[PersonalData]` + global `PiiMaskingResultFilter`.
3. Add `[PersonalData]` to all models from E2; verify every listed page shows masked data.
4. SQL Dynamic Data Masking on plain PII columns.
5. Password hashing + lazy migration + lockout after 5 failed attempts.
6. Move photos/documents outside wwwroot; secure file endpoint.
7. TDE, `Encrypt=True`, HTTPS/HSTS.
8. Serilog: never log request/response bodies of personal-data endpoints; add a destructuring policy that drops `[PersonalData]` properties.

Permission matrix

Permission (existing flag)	Who	Allows
Normal activity mapping	All staff by role	See screens with masked data
`IsPII = 1` on *Student Full Details*	School Admin	Reveal full details, view photos
`IsModifiy = 1` on *Student Full Details*	School Admin	Reset password, unlock account
`IsExportToExcel`	Selected staff	Export (always masked, logged with row count)
DPDP screens (`DPDP.View` / `DPDP.Manage` activities)	Principal, DPO	Dashboard, registers, requests
Breach Register activity	Principal, DPO, IT	Record and report breaches

Step E6 — Audit logging

- [ ] `USP_DPDP_InsertAccessLog` (insert only) and `USP_DPDP_InsertRevealLog` (Part D).
- [ ] `[AuditPersonalData]` filter on every controller returning or changing student, parent or staff data, and every export endpoint (with row count).
- [ ] Logging never breaks the request (try/catch).
- [ ] Access Log Report screen (filter by user, student, action, date).
- [ ] Alert to DPO when one user reveals > 30 records/hour or exports > 1,000 rows/day.

```
CREATE OR ALTER PROCEDURE dbo.USP_DPDP_InsertAccessLog
    @GroupCode INT, @BranchCode INT, @LoginId INT,
    @Action VARCHAR(10), @Entity VARCHAR(50), @EntityId VARCHAR(50) = NULL,
    @RowsAffected INT = NULL, @Fields VARCHAR(500) = NULL,
    @Endpoint VARCHAR(200) = NULL, @IpAddress VARCHAR(45) = NULL
AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO dbo.V3M_DPDP_AccessLog
        (GroupCode, BranchCode, LoginId, Action, Entity, EntityId, RowsAffected, Fields, Endpoint, IpAddress)
    VALUES
        (@GroupCode, @BranchCode, @LoginId, @Action, @Entity, @EntityId, @RowsAffected, @Fields, @Endpoint, @IpAddress);
END
```

```
[AttributeUsage(AttributeTargets.Class | AttributeTargets.Method)]
public class AuditPersonalDataAttribute : TypeFilterAttribute
{
    public AuditPersonalDataAttribute(string entity, string action = "VIEW")
```

```
        : base(typeof(AuditPersonalDataFilter)) => Arguments = new object[] { entity, action };
}

public class AuditPersonalDataFilter : IAsyncActionFilter
{
    private readonly IDpdpAccessLogService _log;
    private readonly string _entity, _action;

    public AuditPersonalDataFilter(IDpdpAccessLogService log, string entity, string action)
    { _log = log; _entity = entity; _action = action; }

    public async Task OnActionExecutionAsync(ActionExecutingContext ctx, ActionExecutionDelegate next)
    {
        await next();
        try
        {
            var http = ctx.HttpContext;
            int C(string n) => int.TryParse(http.User.FindFirst(n)?.Value, out var v) ? v : 0;
            await _log.InsertAsync(new DpdpAccessLogModel
            {
                GroupCode  = C("GroupCode"),
                BranchCode = C("BranchCode"),
                LoginId    = C("LoginId"),
                Action     = _action,
                Entity     = _entity,
                EntityId   = ctx.ActionArguments.TryGetValue("studentId", out var id) ? id?.ToString() : null,
                Endpoint   = http.Request.Path,
                IpAddress  = http.Connection.RemoteIpAddress?.ToString()
            });
        }
        catch { /* logging must never break the request */ }
    }
}
```

Step E7 — Privacy notice and consent capture

- [ ] **Privacy Notice** screen: version, language, text/PDF, contact person, activate.
- [ ] Notice lists data collected, purposes, how to withdraw, how to raise a grievance, rights, contact person. Plain language, separate from the admission form.
- [ ] On **Enquiry, Registration, Admission**: show active notice + checkbox per optional purpose; mandatory purposes shown as information.
- [ ] Save all consents in one call with TVP `TT_DPDP_ConsentItem`.
- [ ] Store notice version, channel, IP and SHA-256 hash.
- [ ] Withdrawal as easy as giving: same screen; sets `WITHDRAWN`.

```
CREATE OR ALTER PROCEDURE dbo.USP_DPDP_SaveConsent
    @GroupCode INT, @BranchCode INT, @SessionId INT = NULL,
    @DataPrincipalType VARCHAR(10), @DataPrincipalId INT,
    @StudentId INT = NULL, @GivenByParentId INT = NULL,
    @NoticeId INT, @Channel VARCHAR(20), @IpAddress VARCHAR(45) = NULL,
    @Items dbo.TT_DPDP_ConsentItem READONLY,
    @CreatedBy INT,
    @Message NVARCHAR(200) OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRAN;

        IF @StudentId IS NOT NULL AND NOT EXISTS (
            SELECT 1 FROM dbo.V3M_DPDP_ParentVerification
            WHERE StudentId = @StudentId AND ParentId = @GivenByParentId
              AND IsAdultConfirmed = 1 AND IsActive = 1)
        BEGIN
            SET @Message = N'Parent is not verified for this student.';
            ROLLBACK; RETURN;
        END

        UPDATE c SET c.Status = 'EXPIRED'
        FROM dbo.V3M_DPDP_Consent c
        JOIN @Items i ON i.PurposeId = c.PurposeId
        WHERE c.GroupCode = @GroupCode AND c.BranchCode = @BranchCode
          AND c.DataPrincipalType = @DataPrincipalType AND c.DataPrincipalId = @DataPrincipalId
          AND c.Status = 'GIVEN';

        INSERT INTO dbo.V3M_DPDP_Consent
            (GroupCode, BranchCode, SessionId, DataPrincipalType, DataPrincipalId, StudentId,
             GivenByParentId, PurposeId, NoticeId, Status, GivenOn, WithdrawnOn, Channel,
             IpAddress, ConsentHash, CreatedBy)
        SELECT @GroupCode, @BranchCode, @SessionId, @DataPrincipalType, @DataPrincipalId, @StudentId,
               @GivenByParentId, i.PurposeId, @NoticeId, i.Status,
               CASE WHEN i.Status = 'GIVEN'     THEN SYSDATETIME() END,
               CASE WHEN i.Status = 'WITHDRAWN' THEN SYSDATETIME() END,
               @Channel, @IpAddress,
               HASHBYTES('SHA2_256', CONCAT(@DataPrincipalType, '|', @DataPrincipalId, '|', i.PurposeId, '|',
                                            @NoticeId, '|', i.Status, '|', CONVERT(VARCHAR(30), SYSDATETIME(), 126))),
               @CreatedBy
        FROM @Items i;

        COMMIT;
```

```
            SET @Message = N'Consent saved.';
        END TRY
        BEGIN CATCH
            IF @@TRANCOUNT > 0 ROLLBACK;
            SET @Message = ERROR_MESSAGE();
        END CATCH
    END
```

```
    var items = new DataTable();
    items.Columns.Add("PurposeId", typeof(int));
    items.Columns.Add("Status", typeof(string));
    foreach (var p in model.Items) items.Rows.Add(p.PurposeId, p.Status);

    var param = new DynamicParameters();
    param.Add("@GroupCode", model.GroupCode);
    param.Add("@BranchCode", model.BranchCode);
    param.Add("@SessionId", model.SessionId);
    param.Add("@DataPrincipalType", model.DataPrincipalType);
    param.Add("@DataPrincipalId", model.DataPrincipalId);
    param.Add("@StudentId", model.StudentId);
    param.Add("@GivenByParentId", model.GivenByParentId);
    param.Add("@NoticeId", model.NoticeId);
    param.Add("@Channel", model.Channel);
    param.Add("@IpAddress", model.IpAddress);
    param.Add("@Items", items.AsTableValuedParameter("dbo.TT_DPDP_ConsentItem"));
    param.Add("@CreatedBy", model.CreatedBy);
    param.Add("@Message", dbType: DbType.String, size: 200, direction: ParameterDirection.Output);

    await conn.ExecuteAsync("dbo.USP_DPDP_SaveConsent", param, commandType: CommandType.StoredProcedure);
    var message = param.Get<string>("@Message");
```

Step E8 — Verifiable parental consent

- [ ] At admission, verify the parent with one method and save in `V3M_DPDP_ParentVerification`:
  - **In person:** staff check an ID and tick "adult confirmed"; store document type + last 4 digits only, never a copy.
  - **OTP + KYC:** OTP to the registered mobile of a parent already verified in school records.
  - **DigiLocker / authorised entity:** store only the returned token / reference (Rule 10).
- [ ] Student with a disability: lawful guardian and appointing order (Rule 11) in `V3M_DPDP_Guardian`.
- [ ] Consent save blocked when the parent is not verified (E7 SP).

> The school exemption covers **only** educational activities and child safety. Promotions, website photos and third-party sharing still need verified parental consent.

Step E9 — Consent check before optional use

```
CREATE OR ALTER PROCEDURE dbo.USP_DPDP_CheckConsent
    @GroupCode INT, @BranchCode INT,
    @DataPrincipalType VARCHAR(10), @DataPrincipalId INT,
    @PurposeCode VARCHAR(30)
AS
BEGIN
    SET NOCOUNT ON;
    SELECT CAST(CASE
             WHEN p.LegalBasis <> 'CONSENT' THEN 1
             WHEN EXISTS (SELECT 1 FROM dbo.V3M_DPDP_Consent c
                          WHERE c.GroupCode = @GroupCode AND c.BranchCode = @BranchCode
                            AND c.DataPrincipalType = @DataPrincipalType
                            AND c.DataPrincipalId = @DataPrincipalId
                            AND c.PurposeId = p.PurposeId AND c.Status = 'GIVEN') THEN 1
             ELSE 0 END AS BIT) AS IsAllowed
    FROM dbo.V3M_DPDP_Purpose p
    WHERE p.GroupCode = @GroupCode AND p.BranchCode = @BranchCode
      AND p.PurposeCode = @PurposeCode AND p.IsActive = 1;
END
```

- [ ] Call it in every promotional SMS / WhatsApp / photo-publishing service before sending.
- [ ] Bulk sends: join `V3M_DPDP_Consent` inside the existing bulk SMS SP instead of calling per parent.
- [ ] Withdrawing `PROMO_SMS` must not stop fee / attendance SMS.

Step E10 — Rights requests, grievances, nominee

Right	What SchoolEye does
Access (s.11)	Summary of data held, purposes, third parties; export from `USP_DPDP_GetPersonalDataSummary` (to the verified parent only)
Correction / update (s.12)	DPO updates the record; change logged; parent told
Erasure (s.12)	Adds to `ErasureQueue` after legal-hold check; statutory records anonymised
Withdraw consent (s.6)	Status `WITHDRAWN`; that purpose stops
Grievance (s.13)	Register, assign, resolve within 90 days; escalation flag
Nominee (s.14)	Nominee details stored, shown to DPO

- [ ] SPs: `USP_DPDP_SaveRightsRequest`, `USP_DPDP_UpdateRightsRequestStatus`, `V3M_DPDP_GetRightsRequestList`, `USP_DPDP_SaveGrievance`, `V3M_DPDP_GetGrievanceList`, `USP_DPDP_SaveNominee`, `USP_DPDP_GetPersonalDataSummary`.
- [ ] Numbers from `V3M_DPDP_Counter` with `UPDLOCK, HOLDLOCK`.
- [ ] `DueOn`: 30 days for requests; grievances `ReceivedOn + 90 days` (computed column).
- [ ] Identity: logged-in parent portal or OTP.
- [ ] Overdue items in red on the dashboard.

```
        DECLARE @NextNo INT;
        UPDATE dbo.V3M_DPDP_Counter WITH (UPDLOCK, HOLDLOCK)
        SET @NextNo = LastNo = LastNo + 1
        WHERE GroupCode = @GroupCode AND BranchCode = @BranchCode AND CounterType = 'GRV';

        IF @@ROWCOUNT = 0
        BEGIN
            INSERT INTO dbo.V3M_DPDP_Counter (GroupCode, BranchCode, CounterType, LastNo)
            VALUES (@GroupCode, @BranchCode, 'GRV', 1);
            SET @NextNo = 1;
        END

        SET @GrievanceNo = CONCAT('GRV/', @BranchCode, '/', RIGHT('00000' + CAST(@NextNo AS VARCHAR(10)), 5));
```

Step E11 — Breach handling

```
Hour 0: school becomes aware ──────────────────────► Hour 72: detailed report due

Detect, record ─► Contain ─► Board: initial ─► Notify parents
(breach register)  (block,     (without delay)   (each affected person)
                   rotate keys)      │                  │
                                     ▼                  ▼
                              Board: detailed      Log notices
                              (cause, fixes)       (channel, sent, delivered)
```

- [ ] Record in Breach Register immediately; `DetectedOn` starts the 72-hour clock.
- [ ] Contain: disable accounts, rotate passwords and **PII encryption keys if exposed** (re-encrypt the vault with `KeyVersion = 2`), block IPs.
- [ ] Initial intimation to the Board without delay: description, nature, extent, timing, location, likely impact.
- [ ] Notify each affected parent / student without delay (SMS / email / app): what happened, likely effects, what the school did, what they should do, contact person. Log in `V3M_DPDP_BreachNotice` (TVP `TT_DPDP_BreachRecipient`).
- [ ] Within 72 hours: detailed report (facts, causes, measures, findings, notices sent). Record extension reference if granted.
- [ ] V3M informs the school immediately if it detects the breach (per DPA).
- [ ] Close with root cause and lessons learned.

Step E12 — Retention and erasure job

- [ ] Nightly job (Hangfire / Windows Service / SQL Agent) runs `USP_DPDP_BuildErasureQueue`: finds people past trigger date + `RetainMonths`, skips legal holds, adds `PENDING` rows.
- [ ] Sends erasure notice by SMS/email; sets `NoticeSentOn` (`NOTIFIED`).
- [ ] After ≥ 48 hours, `USP_DPDP_ExecuteErasure` erases or anonymises; `CHECK` constraint blocks earlier erasure.
- [ ] Anonymise records that must stay (fee receipts: name → `Erased-<id>`; mobile/email/address cleared; **vault row deleted**). Delete photos/documents from storage.
- [ ] Mark `DONE` with `ErasedOn`, `JobRunId`. Admin can set `ON_HOLD` with reason.
- [ ] State backup retention in the notice; erased data disappears as backups rotate.

```
        UPDATE e
        SET e.MobileNo = NULL, e.EmailId = NULL, e.Address = NULL,
            e.StudentName = CONCAT('Erased-', e.EnquiryId)
        FROM dbo.EnquiryMaster e                        -- real table from Step E2
        JOIN dbo.V3M_DPDP_ErasureQueue q ON q.DataPrincipalId = e.EnquiryId
        WHERE q.ErasureQueueId = @ErasureQueueId
          AND q.Status = 'NOTIFIED'
          AND q.NoticeSentOn <= DATEADD(HOUR, -48, SYSDATETIME());

        UPDATE dbo.V3M_DPDP_ErasureQueue
```

```
        SET Status = 'DONE', ErasedOn = SYSDATETIME(), JobRunId = @JobRunId
        WHERE ErasureQueueId = @ErasureQueueId;
```

Step E13 — Blazor screens and parent portal

Screen	Users	Main actions
Student Full Details (Part D)	School Admin	Search, masked profile, reveal with reason, reset password
DPDP Dashboard	Principal, DPO	Consent counts, open requests/grievances with days left, breaches with hours left, erasures due, reveal count today
Purpose & Data Inventory	Admin	Purposes, legal basis, categories, mask rules
Privacy Notice	Admin	Version, language, preview, activate
Consent panel (Admission / Profile)	Office	Notice, optional purposes, parent verification status
Consent Register	DPO	Per-student history; withdraw on request
Rights Requests	DPO	Inbox, assign, respond, attach export
Grievances	DPO	Inbox, 90-day limit, escalate
Breach Register	Principal, DPO, IT	Record, countdown, Board reports, notify parents
Retention & Erasure	Admin	Policies; queue with approve / hold
Access Log Report	Principal, Auditor	Who viewed, revealed or exported whose data
Parent portal: My Privacy	Parents	Notice, consents, request/grievance, nominee

All screens follow the SchoolEye conventions in A.1.

Part F — Testing and go-live

F.1 Masking and reveal tests

✔	Test	Expected
[ ]	Any staff role opens View Student	Mobile `98******21`, email `ra****@…`, Aadhaar `XXXX XXXX 1234`
[ ]	Check API response in browser dev tools / Swagger	Same masked values (not only hidden in UI)
[ ]	Excel export of student list	Masked; log row with row count
[ ]	Role without `IsPII` calls `/api/Dpdp/StudentFullDetails/Reveal` directly	403
[ ]	School Admin reveals without reason	400
[ ]	School Admin reveals with wrong password	400; no data
[ ]	School Admin reveals correctly	Full data incl. Aadhaar, photo; one `UNMASK` log row with reason
[ ]	Wait 120 s	Page re-masks automatically
[ ]	31st reveal within an hour	429; DPO alert
[ ]	Print the page (Ctrl+P)	PII panel not printed
[ ]	Photo URL opened without token	401
[ ]	Photo file path under wwwroot	Not reachable (files moved)
[ ]	Reset password	Temp password shown once; `MustChangePassword = 1`; log row has no password
[ ]	Any API or SP returning a password / hash	None (grep `Password` in all `SELECT` lists)
[ ]	`SELECT` on vault table with SSMS	Only ciphertext
[ ]	Support login without `UNMASK` queries `StudentMaster`	DDM masked values
[ ]	Serilog files after a day of use	No mobile / Aadhaar / email values

F.2 DPDP functional tests

✔	Test	Expected
[ ]	Admission without verified parent	Optional consents cannot be saved
[ ]	Withdraw `PROMO_SMS`	No promo SMS; fee SMS continues
[ ]	UPDATE on access log as API user	Permission denied
[ ]	Erasure < 48 h after notice	Blocked by CHECK constraint
[ ]	Erasure of fee-paying student	Receipts kept, personal fields anonymised, vault row deleted
[ ]	Record breach	Due time = DetectedOn + 72 h on dashboard
[ ]	Grievance older than 90 days	Overdue in red
[ ]	Access request	Parent receives correct summary
[ ]	Penetration / vulnerability test	No high findings open
[ ]	Restore backup	Works; TDE certificate and PII keys available
[ ]	Staff training	Office, accounts, IT trained; attendance recorded
[ ]	Agreements	DPA with each school and third party signed

F.3 Timeline

Phase	Steps	Weeks
Discovery and inventory	E1–E2	1
Governance decisions	E3	1–2 (parallel)
Database, encryption, masking, passwords, audit log	E4–E6	2–5
Student Full Details page	Part D	5–6
Notice, consent, parental verification, consent check	E7–E9	7–9
Rights and grievances	E10	10–11
Breach handling	E11	12
Retention and erasure	E12	13–14
Remaining screens, testing, training, go-live	E13, Part F	15–18

Starting early November 2026, this finishes by **mid-March 2027**, about two months before the May 2027 deadline.

Appendix A — V3M_DPDP_Schema.sql

> Re-runnable. Replace `INT` placeholders for GroupCode / BranchCode / SessionId / StudentId / ParentId / LoginId with the real types from `script.sql`. The PII vault table is in C.3.

```
SET ANSI_NULLS ON;
SET QUOTED_IDENTIFIER ON;
GO

/* ---------- Table types ---------- */
IF TYPE_ID('dbo.TT_DPDP_ConsentItem') IS NULL
    CREATE TYPE dbo.TT_DPDP_ConsentItem AS TABLE
    (
        PurposeId INT         NOT NULL,
        Status    VARCHAR(10) NOT NULL   -- GIVEN / WITHDRAWN
    );
GO

IF TYPE_ID('dbo.TT_DPDP_BreachRecipient') IS NULL
    CREATE TYPE dbo.TT_DPDP_BreachRecipient AS TABLE
    (
        DataPrincipalType VARCHAR(10) NOT NULL,
        DataPrincipalId   INT         NOT NULL,
        Channel           VARCHAR(20) NOT NULL   -- SMS / EMAIL / APP / WHATSAPP
    );
GO

IF TYPE_ID('dbo.TT_DPDP_PiiVaultRow') IS NULL
    CREATE TYPE dbo.TT_DPDP_PiiVaultRow AS TABLE
    (
        OwnerType        VARCHAR(10)    NOT NULL,
        OwnerId          INT            NOT NULL,
        AadhaarEnc       VARBINARY(256) NULL,
        AadhaarLast4     CHAR(4)        NULL,
        AadhaarHash      VARBINARY(32)  NULL,
        BloodGroupEnc    VARBINARY(128) NULL,
        MedicalNotesEnc  VARBINARY(MAX) NULL,
        DisabilityEnc    VARBINARY(512) NULL,
        BankAccountEnc   VARBINARY(256) NULL,
        BankAccountLast4 CHAR(4)        NULL,
        IfscEnc          VARBINARY(128) NULL,
        FamilyIncomeEnc  VARBINARY(128) NULL
    );
GO

/* ---------- Inventory & purposes ---------- */
IF OBJECT_ID('dbo.V3M_DPDP_DataCategory') IS NULL
CREATE TABLE dbo.V3M_DPDP_DataCategory
(
    DataCategoryId INT IDENTITY(1,1) PRIMARY KEY,
    GroupCode      INT          NOT NULL,
    BranchCode     INT          NOT NULL,
    TableName      VARCHAR(128) NOT NULL,
    ColumnName     VARCHAR(128) NOT NULL,
    CategoryCode   VARCHAR(20)  NOT NULL
        CONSTRAINT CK_DPDP_DataCategory_Code CHECK (CategoryCode IN
        ('IDENTITY','DOB','CONTACT','ADDRESS','AADHAAR','HEALTH','PHOTO','FINANCIAL','ACADEMIC','SOCIAL','LOGIN')),
    IsSensitive    BIT          NOT NULL DEFAULT 0,
    IsEncrypted    BIT          NOT NULL DEFAULT 0,
    MaskRule       VARCHAR(100) NULL,
    IsActive       BIT          NOT NULL DEFAULT 1,
    CreatedBy      INT          NOT NULL,
    CreatedOn      DATETIME2(0) NOT NULL DEFAULT SYSDATETIME(),
    CONSTRAINT UQ_DPDP_DataCategory UNIQUE (GroupCode, BranchCode, TableName, ColumnName)
);
GO

IF OBJECT_ID('dbo.V3M_DPDP_Purpose') IS NULL
CREATE TABLE dbo.V3M_DPDP_Purpose
(
    PurposeId       INT IDENTITY(1,1) PRIMARY KEY,
    GroupCode       INT           NOT NULL,
    BranchCode      INT           NOT NULL,
    PurposeCode     VARCHAR(30)   NOT NULL,
    PurposeName     NVARCHAR(200) NOT NULL,
    LegalBasis      VARCHAR(20)   NOT NULL
        CONSTRAINT CK_DPDP_Purpose_Basis CHECK (LegalBasis IN
        ('CONSENT','LEGITIMATE_USE','EXEMPT_4TH_SCH','LEGAL_OBLIGATION')),
    IsChildExempt   BIT           NOT NULL DEFAULT 0,
    IsMandatory     BIT           NOT NULL DEFAULT 0,
    RetentionMonths INT           NULL,
    IsActive        BIT           NOT NULL DEFAULT 1,
    CreatedBy       INT           NOT NULL,
    CreatedOn       DATETIME2(0)  NOT NULL DEFAULT SYSDATETIME(),
    CONSTRAINT UQ_DPDP_Purpose UNIQUE (GroupCode, BranchCode, PurposeCode)
);
GO

IF OBJECT_ID('dbo.V3M_DPDP_PurposeDataCategory') IS NULL
CREATE TABLE dbo.V3M_DPDP_PurposeDataCategory
```

```
(
    PurposeId      INT NOT NULL REFERENCES dbo.V3M_DPDP_Purpose(PurposeId),
    DataCategoryId INT NOT NULL REFERENCES dbo.V3M_DPDP_DataCategory(DataCategoryId),
    CONSTRAINT PK_DPDP_PurposeDataCategory PRIMARY KEY (PurposeId, DataCategoryId)
);
GO

/* ---------- Notice & consent ---------- */
IF OBJECT_ID('dbo.V3M_DPDP_Notice') IS NULL
CREATE TABLE dbo.V3M_DPDP_Notice
(
    NoticeId       INT IDENTITY(1,1) PRIMARY KEY,
    GroupCode      INT            NOT NULL,
    BranchCode     INT            NOT NULL,
    VersionNo      INT            NOT NULL,
    LanguageCode   VARCHAR(10)    NOT NULL DEFAULT 'en',
    NoticeText     NVARCHAR(MAX)  NULL,
    NoticeFilePath NVARCHAR(500)  NULL,
    ContactName    NVARCHAR(150)  NOT NULL,
    ContactEmail   VARCHAR(150)   NOT NULL,
    ContactMobile  VARCHAR(20)    NULL,
    IsActive       BIT            NOT NULL DEFAULT 0,
    ActivatedOn    DATETIME2(0)   NULL,
    CreatedBy      INT            NOT NULL,
    CreatedOn      DATETIME2(0)   NOT NULL DEFAULT SYSDATETIME(),
    CONSTRAINT UQ_DPDP_Notice UNIQUE (GroupCode, BranchCode, LanguageCode, VersionNo)
);
GO

IF OBJECT_ID('dbo.V3M_DPDP_Consent') IS NULL
CREATE TABLE dbo.V3M_DPDP_Consent
(
    ConsentId         BIGINT IDENTITY(1,1) PRIMARY KEY,
    GroupCode         INT          NOT NULL,
    BranchCode        INT          NOT NULL,
    SessionId         INT          NULL,
    DataPrincipalType VARCHAR(10)  NOT NULL
        CONSTRAINT CK_DPDP_Consent_Type CHECK (DataPrincipalType IN ('STUDENT','PARENT','STAFF','ENQUIRY')),
    DataPrincipalId   INT          NOT NULL,
    StudentId         INT          NULL,
    GivenByParentId   INT          NULL,
    PurposeId         INT          NOT NULL REFERENCES dbo.V3M_DPDP_Purpose(PurposeId),
    NoticeId          INT          NOT NULL REFERENCES dbo.V3M_DPDP_Notice(NoticeId),
    Status            VARCHAR(10)  NOT NULL
        CONSTRAINT CK_DPDP_Consent_Status CHECK (Status IN ('GIVEN','WITHDRAWN','EXPIRED')),
    GivenOn           DATETIME2(0) NULL,
    WithdrawnOn       DATETIME2(0) NULL,
    Channel           VARCHAR(20)  NOT NULL,
    IpAddress         VARCHAR(45)  NULL,
    ConsentHash       VARBINARY(32) NULL,
    CreatedBy         INT          NOT NULL,
    CreatedOn         DATETIME2(0) NOT NULL DEFAULT SYSDATETIME()
);
GO
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = 'IX_DPDP_Consent_Lookup')
    CREATE INDEX IX_DPDP_Consent_Lookup ON dbo.V3M_DPDP_Consent
        (GroupCode, BranchCode, DataPrincipalType, DataPrincipalId, PurposeId, Status);
GO

/* ---------- Parental verification & guardian ---------- */
IF OBJECT_ID('dbo.V3M_DPDP_ParentVerification') IS NULL
CREATE TABLE dbo.V3M_DPDP_ParentVerification
(
    VerificationId   INT IDENTITY(1,1) PRIMARY KEY,
    GroupCode        INT          NOT NULL,
    BranchCode       INT          NOT NULL,
    StudentId        INT          NOT NULL,
    ParentId         INT          NOT NULL,
    Method           VARCHAR(15)  NOT NULL
        CONSTRAINT CK_DPDP_PV_Method CHECK (Method IN ('IN_PERSON','OTP_KYC','DIGILOCKER')),
    IdDocType        VARCHAR(30)  NULL,
    IdLast4          CHAR(4)      NULL,
    ReferenceToken   VARCHAR(200) NULL,
    IsAdultConfirmed BIT          NOT NULL DEFAULT 0,
    VerifiedBy       INT          NULL,
    VerifiedOn       DATETIME2(0) NOT NULL DEFAULT SYSDATETIME(),
    IsActive         BIT          NOT NULL DEFAULT 1
);
GO
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = 'IX_DPDP_PV_StudentParent')
    CREATE INDEX IX_DPDP_PV_StudentParent ON dbo.V3M_DPDP_ParentVerification (StudentId, ParentId, IsActive);
GO

IF OBJECT_ID('dbo.V3M_DPDP_Guardian') IS NULL
CREATE TABLE dbo.V3M_DPDP_Guardian
(
    GuardianId     INT IDENTITY(1,1) PRIMARY KEY,
    GroupCode      INT           NOT NULL,
    BranchCode     INT           NOT NULL,
    StudentId      INT           NOT NULL,
    GuardianName   NVARCHAR(150) NOT NULL,
    Relation       NVARCHAR(50)  NULL,
    OrderReference NVARCHAR(100) NULL,
    OrderDate      DATE          NULL,
    OrderIssuedBy  NVARCHAR(150) NULL,
    IsVerified     BIT           NOT NULL DEFAULT 0,
    IsActive       BIT           NOT NULL DEFAULT 1,
```

```
    CreatedBy      INT           NOT NULL,
    CreatedOn      DATETIME2(0)  NOT NULL DEFAULT SYSDATETIME()
);
GO

/* ---------- Rights, grievance, nominee ---------- */
IF OBJECT_ID('dbo.V3M_DPDP_Counter') IS NULL
CREATE TABLE dbo.V3M_DPDP_Counter
(
    GroupCode   INT        NOT NULL,
    BranchCode  INT        NOT NULL,
    CounterType VARCHAR(5) NOT NULL,   -- REQ / GRV / BRC
    LastNo      INT        NOT NULL DEFAULT 0,
    CONSTRAINT PK_DPDP_Counter PRIMARY KEY (GroupCode, BranchCode, CounterType)
);
GO

IF OBJECT_ID('dbo.V3M_DPDP_RightsRequest') IS NULL
CREATE TABLE dbo.V3M_DPDP_RightsRequest
(
    RequestId         INT IDENTITY(1,1) PRIMARY KEY,
    RequestNo         VARCHAR(30)    NOT NULL,
    GroupCode         INT            NOT NULL,
    BranchCode        INT            NOT NULL,
    DataPrincipalType VARCHAR(10)    NOT NULL,
    DataPrincipalId   INT            NOT NULL,
    RequestType       VARCHAR(15)    NOT NULL
        CONSTRAINT CK_DPDP_RR_Type CHECK (RequestType IN ('ACCESS','CORRECTION','ERASURE','UPDATE')),
    Details           NVARCHAR(1000) NULL,
    Channel           VARCHAR(20)    NOT NULL,
    VerifiedVia       VARCHAR(20)    NULL,
    Status            VARCHAR(12)    NOT NULL DEFAULT 'OPEN'
        CONSTRAINT CK_DPDP_RR_Status CHECK (Status IN ('OPEN','IN_PROGRESS','CLOSED','REJECTED')),
    ReceivedOn        DATETIME2(0)   NOT NULL DEFAULT SYSDATETIME(),
    DueOn             DATETIME2(0)   NOT NULL,
    AssignedTo        INT            NULL,
    ResponseText      NVARCHAR(2000) NULL,
    ResponseFilePath  NVARCHAR(500)  NULL,
    ClosedOn          DATETIME2(0)   NULL,
    CreatedBy         INT            NOT NULL,
    CONSTRAINT UQ_DPDP_RR_No UNIQUE (GroupCode, BranchCode, RequestNo)
);
GO

IF OBJECT_ID('dbo.V3M_DPDP_Grievance') IS NULL
CREATE TABLE dbo.V3M_DPDP_Grievance
(
    GrievanceId       INT IDENTITY(1,1) PRIMARY KEY,
    GrievanceNo       VARCHAR(30)    NOT NULL,
    GroupCode         INT            NOT NULL,
    BranchCode        INT            NOT NULL,
    DataPrincipalType VARCHAR(10)    NOT NULL,
    DataPrincipalId   INT            NOT NULL,
    Category          VARCHAR(30)    NULL,
    Description       NVARCHAR(2000) NOT NULL,
    Channel           VARCHAR(20)    NOT NULL,
    Status            VARCHAR(12)    NOT NULL DEFAULT 'OPEN'
        CONSTRAINT CK_DPDP_GRV_Status CHECK (Status IN ('OPEN','IN_PROGRESS','RESOLVED','REJECTED')),
    ReceivedOn        DATETIME2(0)   NOT NULL DEFAULT SYSDATETIME(),
    DueOn             AS DATEADD(DAY, 90, ReceivedOn) PERSISTED,
    AssignedTo        INT            NULL,
    IsEscalated       BIT            NOT NULL DEFAULT 0,
    Resolution        NVARCHAR(2000) NULL,
    ResolvedOn        DATETIME2(0)   NULL,
    CreatedBy         INT            NOT NULL,
    CONSTRAINT UQ_DPDP_GRV_No UNIQUE (GroupCode, BranchCode, GrievanceNo)
);
GO

IF OBJECT_ID('dbo.V3M_DPDP_Nominee') IS NULL
CREATE TABLE dbo.V3M_DPDP_Nominee
(
    NomineeId         INT IDENTITY(1,1) PRIMARY KEY,
    GroupCode         INT           NOT NULL,
    BranchCode        INT           NOT NULL,
    DataPrincipalType VARCHAR(10)   NOT NULL,
    DataPrincipalId   INT           NOT NULL,
    NomineeName       NVARCHAR(150) NOT NULL,
    Relation          NVARCHAR(50)  NULL,
    NomineeMobile     VARCHAR(20)   NULL,
    NomineeEmail      VARCHAR(150)  NULL,
    IsActive          BIT           NOT NULL DEFAULT 1,
    CreatedBy         INT           NOT NULL,
    CreatedOn         DATETIME2(0)  NOT NULL DEFAULT SYSDATETIME()
);
GO

/* ---------- Breach ---------- */
IF OBJECT_ID('dbo.V3M_DPDP_Breach') IS NULL
CREATE TABLE dbo.V3M_DPDP_Breach
(
    BreachId             INT IDENTITY(1,1) PRIMARY KEY,
    BreachNo             VARCHAR(30)    NOT NULL,
    GroupCode            INT            NOT NULL,
    BranchCode           INT            NOT NULL,
    Title                NVARCHAR(200)  NOT NULL,
    Description          NVARCHAR(MAX)  NULL,
```

```
    Nature              NVARCHAR(200)  NULL,
    Location            NVARCHAR(200)  NULL,
    OccurredFrom        DATETIME2(0)   NULL,
    OccurredTo          DATETIME2(0)   NULL,
    DetectedOn          DATETIME2(0)   NOT NULL,
    AffectedCount       INT            NULL,
    LikelyImpact        NVARCHAR(1000) NULL,
    ContainmentActions  NVARCHAR(MAX)  NULL,
    ReportedByProcessor BIT            NOT NULL DEFAULT 0,
    BoardInitialSentOn  DATETIME2(0)   NULL,
    BoardDetailedDueOn  AS DATEADD(HOUR, 72, DetectedOn) PERSISTED,
    BoardDetailedSentOn DATETIME2(0)   NULL,
    BoardExtensionRef   NVARCHAR(100)  NULL,
    RootCause           NVARCHAR(MAX)  NULL,
    LessonsLearned      NVARCHAR(MAX)  NULL,
    Status              VARCHAR(12)    NOT NULL DEFAULT 'OPEN'
        CONSTRAINT CK_DPDP_Breach_Status CHECK (Status IN ('OPEN','CONTAINED','REPORTED','CLOSED')),
    CreatedBy           INT            NOT NULL,
    CreatedOn           DATETIME2(0)   NOT NULL DEFAULT SYSDATETIME(),
    CONSTRAINT UQ_DPDP_Breach_No UNIQUE (GroupCode, BranchCode, BreachNo)
);
GO

IF OBJECT_ID('dbo.V3M_DPDP_BreachNotice') IS NULL
CREATE TABLE dbo.V3M_DPDP_BreachNotice
(
    BreachNoticeId    BIGINT IDENTITY(1,1) PRIMARY KEY,
    BreachId          INT          NOT NULL REFERENCES dbo.V3M_DPDP_Breach(BreachId),
    DataPrincipalType VARCHAR(10)  NOT NULL,
    DataPrincipalId   INT          NOT NULL,
    Channel           VARCHAR(20)  NOT NULL,
    SentOn            DATETIME2(0) NULL,
    DeliveryStatus    VARCHAR(15)  NOT NULL DEFAULT 'PENDING',
    CreatedOn         DATETIME2(0) NOT NULL DEFAULT SYSDATETIME()
);
GO

/* ---------- Retention & erasure ---------- */
IF OBJECT_ID('dbo.V3M_DPDP_RetentionPolicy') IS NULL
CREATE TABLE dbo.V3M_DPDP_RetentionPolicy
(
    PolicyId       INT IDENTITY(1,1) PRIMARY KEY,
    GroupCode      INT           NOT NULL,
    BranchCode     INT           NOT NULL,
    DataCategoryId INT           NULL REFERENCES dbo.V3M_DPDP_DataCategory(DataCategoryId),
    PolicyName     NVARCHAR(150) NOT NULL,
    TriggerEvent   VARCHAR(30)   NOT NULL,
    RetainMonths   INT           NULL,
    ActionType     VARCHAR(10)   NOT NULL
        CONSTRAINT CK_DPDP_RP_Action CHECK (ActionType IN ('ERASE','ANONYMISE','KEEP')),
    IsActive       BIT           NOT NULL DEFAULT 1,
    CreatedBy      INT           NOT NULL,
    CreatedOn      DATETIME2(0)  NOT NULL DEFAULT SYSDATETIME()
);
GO

IF OBJECT_ID('dbo.V3M_DPDP_ErasureQueue') IS NULL
CREATE TABLE dbo.V3M_DPDP_ErasureQueue
(
    ErasureQueueId    BIGINT IDENTITY(1,1) PRIMARY KEY,
    GroupCode         INT           NOT NULL,
    BranchCode        INT           NOT NULL,
    PolicyId          INT           NOT NULL REFERENCES dbo.V3M_DPDP_RetentionPolicy(PolicyId),
    DataPrincipalType VARCHAR(10)   NOT NULL,
    DataPrincipalId   INT           NOT NULL,
    TriggerDate       DATE          NOT NULL,
    DueOn             DATE          NOT NULL,
    Source            VARCHAR(15)   NOT NULL DEFAULT 'RETENTION',
    Status            VARCHAR(10)   NOT NULL DEFAULT 'PENDING'
        CONSTRAINT CK_DPDP_EQ_Status CHECK (Status IN ('PENDING','NOTIFIED','DONE','ON_HOLD','FAILED')),
    NoticeSentOn      DATETIME2(0)  NULL,
    ErasedOn          DATETIME2(0)  NULL,
    HoldReason        NVARCHAR(500) NULL,
    JobRunId          VARCHAR(50)   NULL,
    CreatedOn         DATETIME2(0)  NOT NULL DEFAULT SYSDATETIME(),
    CONSTRAINT CK_DPDP_EQ_48h CHECK
        (ErasedOn IS NULL OR (NoticeSentOn IS NOT NULL AND ErasedOn >= DATEADD(HOUR, 48, NoticeSentOn)))
);
GO

/* ---------- Access log (insert-only) ---------- */
IF OBJECT_ID('dbo.V3M_DPDP_AccessLog') IS NULL
CREATE TABLE dbo.V3M_DPDP_AccessLog
(
    AccessLogId  BIGINT IDENTITY(1,1) PRIMARY KEY,
    GroupCode    INT           NOT NULL,
    BranchCode   INT           NOT NULL,
    LoginId      INT           NOT NULL,
    Action       VARCHAR(10)   NOT NULL,   -- VIEW / EDIT / EXPORT / UNMASK / RESET_PWD / DELETE
    Entity       VARCHAR(50)   NOT NULL,   -- STUDENT / PARENT / STAFF / ENQUIRY
    EntityId     VARCHAR(50)   NULL,
    RowsAffected INT           NULL,
    Fields       VARCHAR(500)  NULL,
    Endpoint     VARCHAR(200)  NULL,
    IpAddress    VARCHAR(45)   NULL,
    ReasonCode   VARCHAR(30)   NULL,
    ReasonText   NVARCHAR(300) NULL,
```

```
    LoggedOn     DATETIME2(0)  NOT NULL DEFAULT SYSDATETIME()
);
GO
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = 'IX_DPDP_AccessLog_Entity')
    CREATE INDEX IX_DPDP_AccessLog_Entity ON dbo.V3M_DPDP_AccessLog (GroupCode, BranchCode, Entity, EntityId, LoggedOn);
GO
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = 'IX_DPDP_AccessLog_LoginAction')
    CREATE INDEX IX_DPDP_AccessLog_LoginAction ON dbo.V3M_DPDP_AccessLog (LoginId, Action, LoggedOn);
GO
-- Run once with the real API login/user name:
-- DENY UPDATE, DELETE ON dbo.V3M_DPDP_AccessLog TO [SchoolEyeApiUser];

/* ---------- Processors / third parties ---------- */
IF OBJECT_ID('dbo.V3M_DPDP_Processor') IS NULL
CREATE TABLE dbo.V3M_DPDP_Processor
(
    ProcessorId       INT IDENTITY(1,1) PRIMARY KEY,
    GroupCode         INT           NOT NULL,
    BranchCode        INT           NOT NULL,
    ProcessorName     NVARCHAR(150) NOT NULL,
    ServiceType       VARCHAR(30)   NOT NULL,
    DataShared        NVARCHAR(500) NULL,
    AgreementSignedOn DATE          NULL,
    AgreementFilePath NVARCHAR(500) NULL,
    ContactEmail      VARCHAR(150)  NULL,
    IsActive          BIT           NOT NULL DEFAULT 1,
    CreatedBy         INT           NOT NULL,
    CreatedOn         DATETIME2(0)  NOT NULL DEFAULT SYSDATETIME()
);
GO
```

Appendix B — Ready-to-paste Claude Code prompts

Run one phase at a time. Review and commit after each.

Phase 1 — Discovery

```
Read docs/DPDP_IMPLEMENTATION.md fully. Do Step E1 and Step E2 only.
Do not change any code. Produce docs/DPDP_DISCOVERY.md with: project list, auth and JWT
claim names, permission model, real types of GroupCode/BranchCode/SessionId/StudentId/
ParentId/LoginId, the filled placeholder table from A.2, how passwords are stored and
verified, where photos/documents are stored, every SMS/WhatsApp/email sending point,
the data inventory from the E2 query, and the list of models that need [PersonalData].
Stop and wait for my approval.
```

Phase 2 — Database objects

```
Following docs/DPDP_IMPLEMENTATION.md and docs/DPDP_DISCOVERY.md, create
Database/DPDP/01_V3M_DPDP_Schema.sql from Appendix A plus the vault table from C.3,
with the real column types. Re-runnable. Do not run it. Show me the file.
```

Phase 3 — Encryption and masking

```
Implement Part C sections C.3 and C.4 only: IPiiCryptoService, vault migration console
job, IMaskingService, [PersonalData], global PiiMaskingResultFilter. Add [PersonalData]
to every model listed in DPDP_DISCOVERY.md. Follow rules A.1. Do not change controllers.
List every model and property you changed.
```

Phase 4 — Passwords, files, DDM, logging

```
Implement C.5 (DDM script), C.7 (password hashing with lazy migration and lockout),
moving photos/documents outside wwwroot with a secure endpoint, Step E6 audit logging,
and the Serilog rule from E5. One file set per API as per A.1 rule 12.
```

Phase 5 — Student Full Details page

```
Implement Part D completely: menu/permission SQL (D.2), the five SPs (D.3), models (D.4),
service (D.5), controller (D.6) and the Blazor page (D.7) using the existing PageHeading,
search popup, QuickGrid and SweetAlert patterns. Use real table names from
DPDP_DISCOVERY.md. Then walk through every row of test table F.1 and tell me the result.
```

Phases 6–9

```
Implement Step E7 + E8 (notice, consent, parent verification).
Implement Step E9 (consent check in SMS/WhatsApp, including bulk SMS SP).
Implement Step E10 + E11 (rights, grievances, nominee, breach register).
Implement Step E12 + E13 (erasure job, remaining screens), then run Part F.
```

Sources

- PIB — DPDP Rules, 2025 notified (explainer): https://static.pib.gov.in/WriteReadData/specificdocs/documents/2025/nov/doc20251117695301.pdf
- S.S. Rana & Co. — MeitY notifies final DPDP Rules 2025 (G.S.R. 846(E), 13 Nov 2025; compliance by 13 May 2027): https://ssrana.in/articles/meity-notifies-final-digital-personal-data-protection-rules-2025/
- AZB & Partners — staggered commencement of the DPDP Act and Rules: https://www.azbpartners.com/?p=87799
- Mondaq — DPDP Act compliance mandate (13 Nov 2026 and 13 May 2027 milestones): https://www.mondaq.com/india/privacy-protection/1708830/dpdp-act-compliance-mandate
- DPDP Rules, Rule 7 (breach intimation): https://www.dpdpa.com/dpdparules/rule7.html
- DPDP Rules, Fourth Schedule: https://dpdp.myndsolution.com/wiki/rules/schedule-4-classes-of-data-fiduciaries-in-respect-of-whom-provisions-of/
- Penalty amounts: Schedule to the DPDP Act, 2023 (confirm in the Gazette text)

Technical implementation plan, not legal advice. The school's counsel should confirm the notice text, retention periods, Aadhaar storage and use of exemptions.